

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Đồ án 2: Xây dựng Đám mây Riêng trên nền tảng OpenStack

Môn học: Triển khai và Vận hành Hạ tầng Điện toán Đám mây

Sinh viên thực hiện:

Ngô Tấn Tài – 23127470

Nguyễn Tấn Lộc – 23127406

Hồ Minh Dũng – 23127174

Giảng viên hướng dẫn:

Cô Chung Thùy Linh

Thầy Lê Ngọc Sơn

Thành phố Hồ Chí Minh, Tháng 8/2026

Mục lục

1	Tổng quan & Thông số Môi trường Thực nghiệm	1
1.1	Mục tiêu & Phạm vi Đồ án	1
1.2	Môi trường Thực nghiệm Lab	1
1.3	Phân công Trách nhiệm & Quy tắc Cô lập Tài nguyên	2
2	Kiến trúc Hệ thống & Cấu trúc Tô-pô Mạng	3
2.1	Phân tầng Kiến trúc OpenStack	3
2.2	Kiến trúc Mạng Phân tầng (Multi-Tier Topology)	4
3	Module 1: Bảng điều khiển Horizon & Vòng đời Dịch vụ Cốt lõi	6
3.1	Mục tiêu & Khảo sát Ban đầu	6
3.2	Kiểm tra Hiện trạng Dịch vụ Cốt lõi	6
3.3	Vòng đời Máy ảo: Khởi tạo, Kiểm tra Runtime & Hủy Cấp phát	14
4	Module 2: Cô lập Mạng Đa tầng cho Tenant	19
4.1	Mục tiêu & Nguyên lý Thiết kế	19
4.2	Quy trình Cấu hình Thực tế	19
4.3	Kiểm tra & Phân tích Tô-pô Mạng	20
5	Module 3: Máy ảo Đa Card mạng & Xử lý Định tuyến Bất đối xứng	23
5.1	Mục tiêu & Thách thức Kiến trúc Định tuyến	23
5.2	Quy trình Triển khai	23
5.3	Cấu hình Bảng Định tuyến Kernel & Kiểm thử Dịch vụ Web	25
6	Module 4: Lưu trữ Bền vững & Phục hồi Thảm họa với Snapshot	28
6.1	Mục tiêu & Mô hình Lưu trữ Lai (Hybrid Storage)	28
6.2	Thiết lập Swift Object Storage	28
6.3	Tích hợp Cinder Block Volume & Triển khai Dịch vụ Web	29
6.4	Tạo Snapshot & Xác thực Phục hồi Thảm họa (Disaster Recovery)	30

7	Module 5: Tự động hóa Khai báo Hạ tầng với OpenStack Heat	32
7.1	Mục tiêu & Mô hình Hạ tầng dưới dạng Mã (IaC)	32
7.2	Cấu trúc Mẫu Điều phối Heat (HOT Architecture)	32
7.3	Pipeline Tự Cấu hình với Cloud-Init	34
7.4	Triển khai Stack & Xác thực Tự động	34
8	Nhật ký Sự cố Kỹ thuật & Phân tích Nguyên nhân Gốc	36
8.1	Sự cố 1: Lỗi HTTP 403 CSRF Forbidden trên Dashboard Horizon	36
8.2	Sự cố 2: Rớt gói tin do Định tuyến Bất đối xứng trên Máy ảo Dual-NIC	36
8.3	Sự cố 3: Máy ảo không thể kết nối tới Swift API từ Subnet Private	37
8.4	Sự cố 4: Race Condition khi Đính kèm Ổ đĩa VirtIO trong Cloud-Init	37
9	Tổng kết & Đánh giá Đồ án	39
9.1	So sánh Vận hành Thủ công vs Tự động hóa Heat	39
9.2	Tổng kết Kết quả Đạt được	39
9.3	Bài học Kinh nghiệm & Hướng Phát triển	40

1 Tổng quan & Thông số Môi trường Thực nghiệm

1.1 Mục tiêu & Phạm vi Đồ án

Trong khuôn khổ đồ án môn học, nhóm chúng em tập trung triển khai và vận hành một hệ thống hạ tầng đám mây riêng (Private Cloud) hoàn chỉnh dựa trên nền tảng mã nguồn mở OpenStack thông qua công cụ tự động hóa DevStack. Mục tiêu cốt lõi của nhóm là trực tiếp khảo sát, thực hành và làm chủ các nguyên lý kỹ thuật hạ tầng đám mây (Cloud Engineering), bao gồm mô hình Hạ tầng dưới dạng Dịch vụ (IaaS), công nghệ mạng ảo hóa điều khiển bằng phần mềm (SDN), quản lý vòng đời lưu trữ khối/đối tượng và tự động hóa điều phối hạ tầng dưới dạng mã (IaC).

Quá trình thực nghiệm của nhóm được chia thành 5 phân hệ kỹ thuật (Module) cụ thể:

- **Dịch vụ cốt lõi & Quản lý vòng đời (Module 1):** Khảo sát, kiểm thử và vận hành các dịch vụ OpenStack thông qua giao diện Web Horizon và công cụ dòng lệnh OpenStack CLI.
- **Phân đoạn & Cô lập mạng Tenant (Module 2):** Xây dựng cấu trúc mạng đa tầng phân lập gồm mạng ngoại vi (Provider Network), vùng mạng DMZ, mạng nội bộ Private và bộ định tuyến ảo L3 Router.
- **Máy ảo đa Card mạng & Cung cấp dịch vụ (Module 3):** Khởi tạo máy ảo gắn đồng thời 2 card mạng (Dual-NIC), xử lý triệt để bài toán định tuyến bất đối xứng (Asymmetric Routing) trong nhân Linux, gán Floating IP và triển khai dịch vụ web công khai.
- **Lưu trữ Bền vững & Phục hồi sau thảm họa (Module 4):** Quản trị tài nguyên tĩnh phi cấu trúc bằng Swift Object Storage và ổ đĩa khối bền vững với Cinder Block Storage; thực hiện quy trình tạo Snapshot và khôi phục dữ liệu nguyên vẹn trên máy ảo độc lập.
- **Tự động hóa Khai báo Hạ tầng với Heat (Module 5):** Soạn thảo mẫu điều phối Heat Orchestration Template (HOT) kết hợp kịch bản Cloud-Init để tự động hóa toàn bộ vòng đời từ cấp phát hạ tầng đến khởi chạy ứng dụng web.

1.2 Môi trường Thực nghiệm Lab

Để phục vụ quá trình thực nghiệm, nhóm chúng em triển khai hệ thống OpenStack theo mô hình All-in-One trên một máy ảo chạy trong môi trường ảo hóa lồng nhau (Nested KVM) thuộc máy chủ Proxmox VE. Thông số cấu hình chi tiết được tổng hợp tại Bảng 1.

Thông số	Cấu hình / Giá trị thực tế
Nền tảng ảo hóa	Proxmox VE (Nested Hardware Virtualization KVM) [1]
Hệ điều hành Host	Ubuntu Server 24.04.4 LTS (noble), Linux Kernel 6.8
Tài nguyên phần cứng	4 vCPUs, 22 GiB RAM, 73 GiB NVMe Root Disk, 8 GiB Swap
Nhánh DevStack	stable/2026.1 (Commit: da2f4d73f5ad74fc8ecfbe15bd7e20f6b0982dbb)
Kiểu ảo hóa Hypervisor	LIBVIRT_TYPE=kvm (Tăng tốc phần cứng QEMU/KVM)
Dịch vụ mở rộng kích hoạt	Heat Orchestration, Heat Dashboard, Swift Object Storage
IP Host / Cổng kết nối	192.168.2.10 / NetBird Overlay Mesh VPN Gateway
Giao diện quản trị Web	https://openstack.net.selfhost.io.vn/dashboa rd

Bảng 1: Thông số môi trường máy chủ và cấu hình triển khai DevStack.

Giải trình kỹ thuật về phiên bản Hệ điều hành: Mặc dù tài liệu đề bài ban đầu gợi ý Ubuntu 22.04 LTS, nhóm chúng em quyết định lựa chọn nền tảng Ubuntu 24.04.4 LTS (noble). Quyết định này bắt nguồn từ yêu cầu kỹ thuật thực tế của nhánh DevStack chính thức mới nhất (stable/2026.1), vốn đòi hỏi môi trường Python 3.12 và các gói thư viện OpenStack phụ thuộc chỉ được hỗ trợ toàn diện trên phiên bản Ubuntu 24.04.

1.3 Phân công Trách nhiệm & Quy tắc Cô lập Tài nguyên

Để đảm bảo quá trình thực hành nhóm diễn ra nhịp nhàng, tránh xung đột tài nguyên trên máy chủ DevStack All-in-One dùng chung, nhóm chúng em đã thống nhất bảng phân công nhiệm vụ cụ thể (Bảng 2) cùng các quy tắc vận hành chặt chẽ.

Thành viên	Vai trò	Nhiệm vụ Phụ trách	Đóng góp
Ngô Tấn Tài (23127470)	Trưởng nhóm		100%
Nguyễn Tấn Lộc (23127406)	Thành viên		100%
Hồ Minh Dũng (23127174)	Thành viên		100%

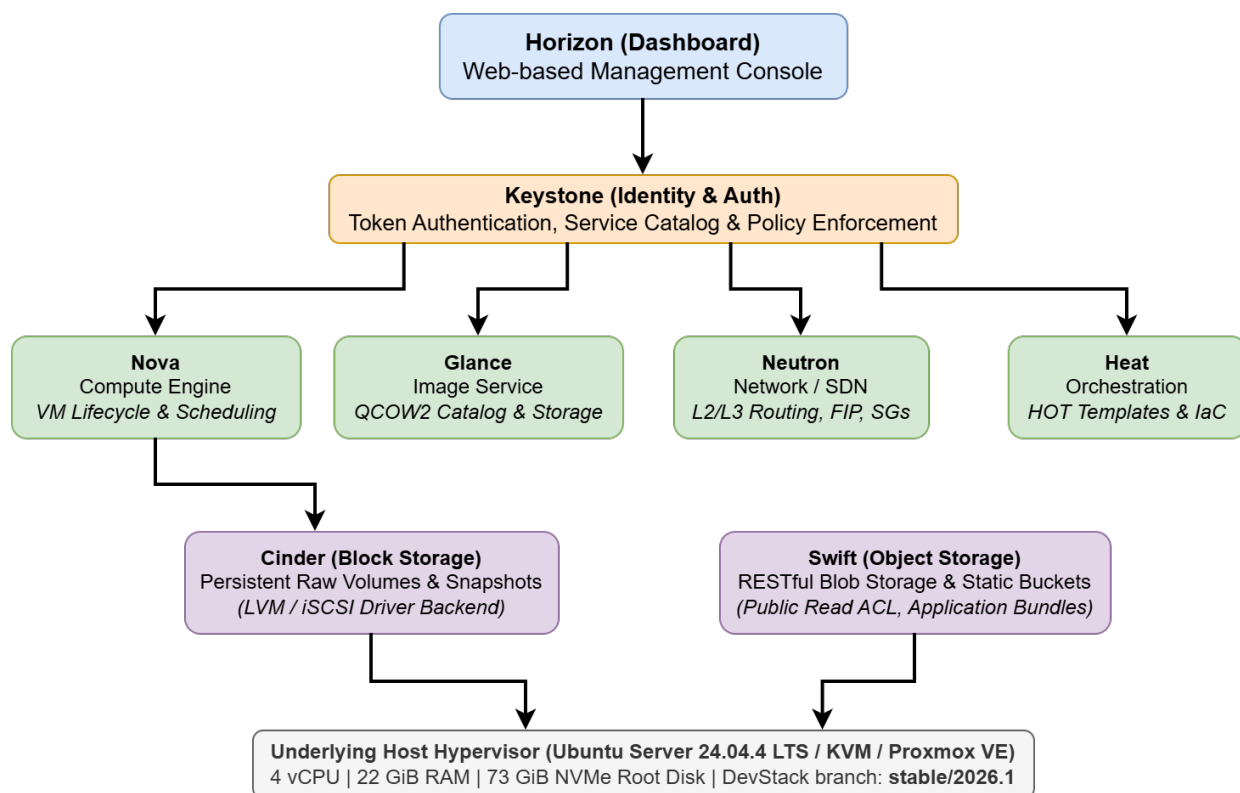
Bảng 2: Bảng phân công trách nhiệm và tỷ lệ đóng góp của các thành viên trong nhóm.

Quy tắc đặt tên tài nguyên (Naming Conventions): Các thành viên trong nhóm thống nhất bắt buộc sử dụng tiền tố theo từng module (ví dụ: m2-dmz-net, m3-web-server, m4-data-volume, m5-server). Quy tắc này giúp ngăn ngừa triệt để việc ghi đè cấu hình và giúp quá trình nghiệm thu, dọn dẹp tài nguyên diễn ra độc lập mà không ảnh hưởng đến phần việc của nhau.

2 Kiến trúc Hệ thống & Cấu trúc Tô-pô Mạng

2.1 Phân tầng Kiến trúc OpenStack

Hệ thống OpenStack vận hành theo mô hình kiến trúc phân tán dạng hướng dịch vụ (Microservices), trong đó các dịch vụ độc lập giao tiếp với nhau thông qua giao diện lập trình ứng dụng RESTful HTTP APIs và hàng đợi thông điệp RabbitMQ, dưới sự chứng thực tập trung của dịch vụ định danh Keystone [2, 3] (Hình 1).



Hình 1: Kiến trúc phân tầng và luồng tương tác giữa các vi dịch vụ trong hệ thống OpenStack.

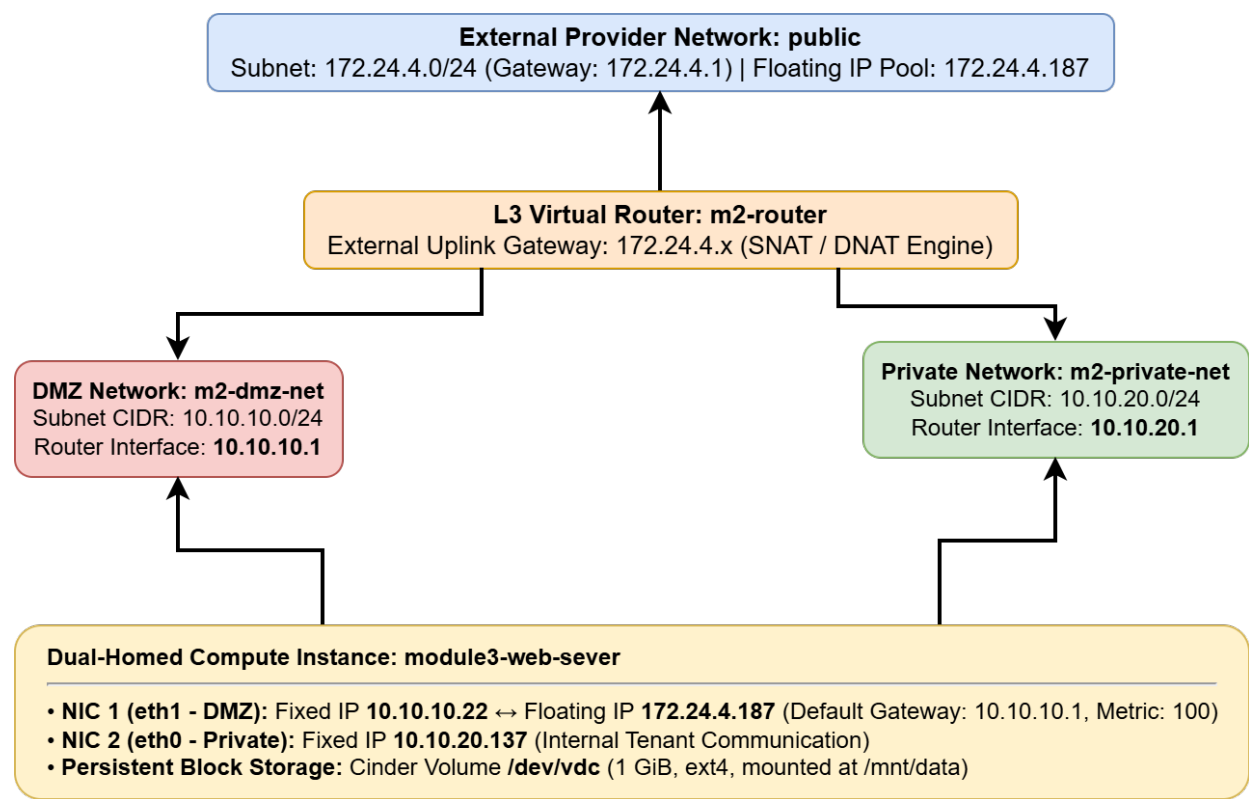
Trong phạm vi triển khai của đề án, các khối dịch vụ cốt lõi đảm nhận các vai trò kỹ thuật cụ thể như sau:

1. **Xác thực & Định danh (Keystone):** Xác thực các yêu cầu API, phát hành mã token có phạm vi truy cập (Scoped Token) và phân giải URL endpoint của các dịch vụ trong mạng nội bộ cũng như mạng quản trị.
2. **Quản lý Điện toán (Nova):** Tương tác với hệ thống quản lý ảo hóa libvirt/qemu để quản lý vòng đời máy ảo, lập lịch thực thi trên hypervisor và cấp phát đĩa gốc (Ephemeral Root Disk).

3. **Quản lý Ảnh máy ảo (Glance):** Lưu trữ và phân phối các tệp ảnh hệ điều hành tương thích với Cloud-Init (`cirros-0.6.3-x86_64-disk` và `Fedora-Cloud-Base-37-1.7`).
4. **Mạng ảo hóa phần mềm (Neutron):** Quản lý hạ tầng mạng ảo SDN [4], các phân đoạn mạng tenant, bộ định tuyến ảo L3 Router và tường lửa phân tán (Security Groups).
5. **Hệ thống Lưu trữ (Cinder & Swift):**
 - **Cinder** [5]: Cung cấp các khối lưu trữ bền vững (Block Storage) gắn vào máy ảo qua giao thức iSCSI/LVM.
 - **Swift** [6]: Cung cấp dịch vụ lưu trữ đối tượng (Object Storage) chuẩn REST API phục vụ lưu trữ file tĩnh và mã nguồn ứng dụng.
6. **Tự động hóa Điều phối (Heat):** Phân tích cú pháp các tệp mẫu khai báo YAML (HOT specification) [7], xây dựng đồ thị có hướng không chu trình (DAG) để tự động khởi tạo hạ tầng theo đúng thứ tự phụ thuộc.

2.2 Kiến trúc Mạng Phân tầng (Multi-Tier Topology)

Hình 2 thể hiện sơ đồ tô-pô mạng logic mà nhóm chúng em đã thiết kế và triển khai xuyên suốt từ Module 2 đến Module 4, mô phỏng cấu trúc mạng phân tầng bảo mật cho doanh nghiệp tuân thủ theo chuẩn RFC 1918 [8].



Hình 2: Tô-pô mạng logic thể hiện sự cô lập đa tầng, định tuyến L3 và kết nối máy ảo đa card mạng.

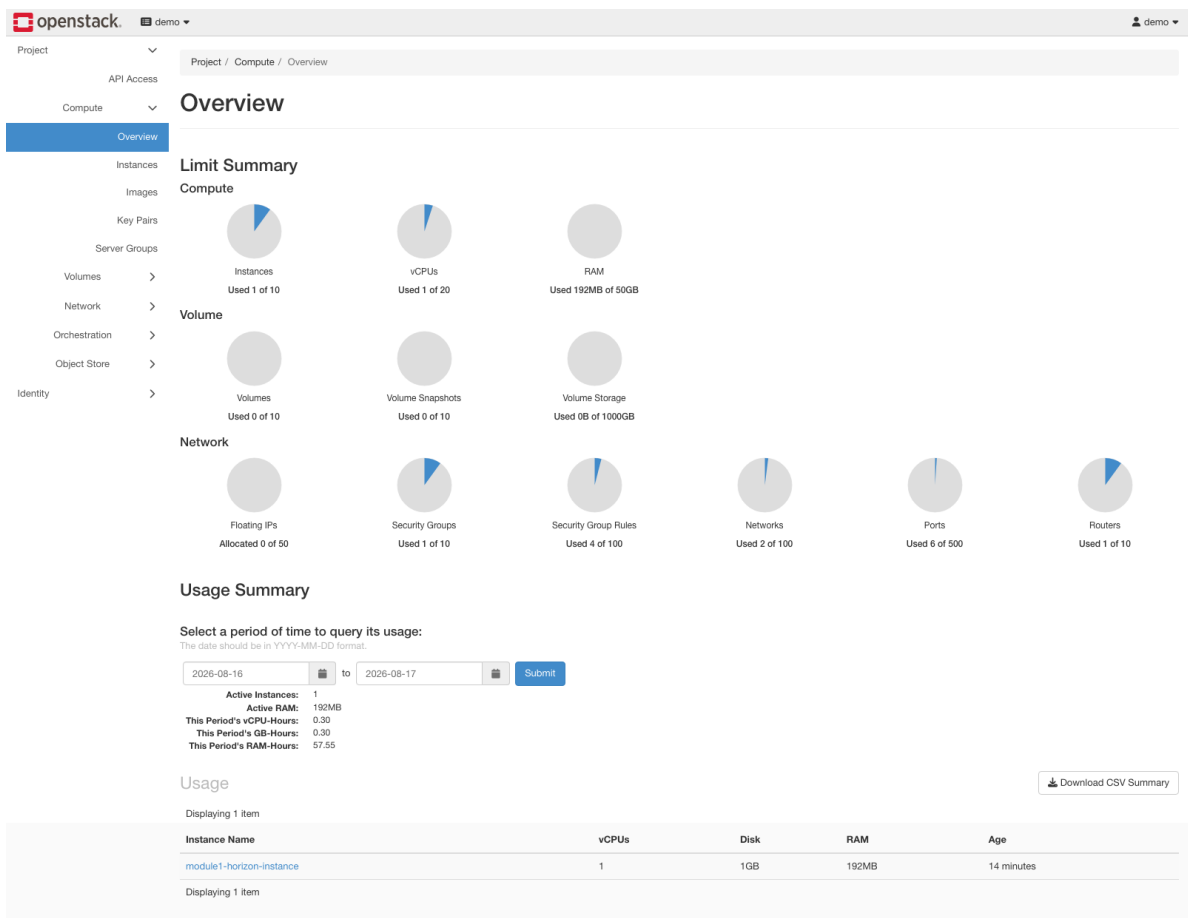
3 Module 1: Bảng điều khiển Horizon & Vòng đời Dịch vụ Cốt lõi

3.1 Mục tiêu & Khảo sát Ban đầu

Trong Module 1, nhóm chúng em tiến hành kiểm tra và xác nhận trạng thái hoạt động ban đầu của toàn bộ nền tảng OpenStack ngay sau khi hoàn tất quá trình cài đặt bằng DevStack. Mục tiêu của bước này là thiết lập đường cơ sở (Baseline) tin cậy cho các lớp dịch vụ điện toán, lưu trữ và mạng, đồng thời làm quen với các thao tác quản trị trên cả giao diện đồ họa Horizon lẫn công cụ dòng lệnh OpenStack CLI.

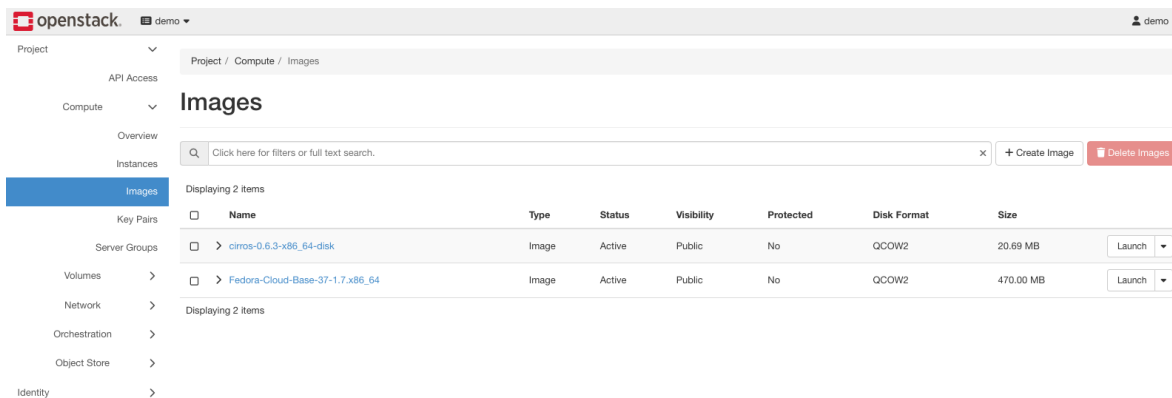
3.2 Kiểm tra Hiện trạng Dịch vụ Cốt lõi

Đăng nhập vào bảng điều khiển Horizon dưới ngữ cảnh dự án demo, nhóm chúng em đã tiến hành khảo sát toàn diện hiện trạng của các dịch vụ:



Hình 3: Giao diện Compute Overview và tổng quan hạn ngạch tài nguyên dự án.

- **Tổng quan Điện toán (Hình 3):** Nhóm ghi nhận hạn ngạch (Quota) mặc định của dự án gồm 10 Instances, 20 vCPUs, 50 GiB RAM, 1000 GiB Volume Storage và 50 Floating IPs.
- **Danh mục Ảnh máy ảo (Hình 4):** Dịch vụ Glance đã tích hợp sẵn hai ảnh máy ảo phục vụ thử nghiệm: bản CirrOS siêu nhẹ `cirros-0.6.3-x86_64-disk` (20.69 MB) để kiểm thử nhanh và bản `Fedora-Cloud-Base-37-1.7` (470 MB) cho các tác vụ Linux đầy đủ.



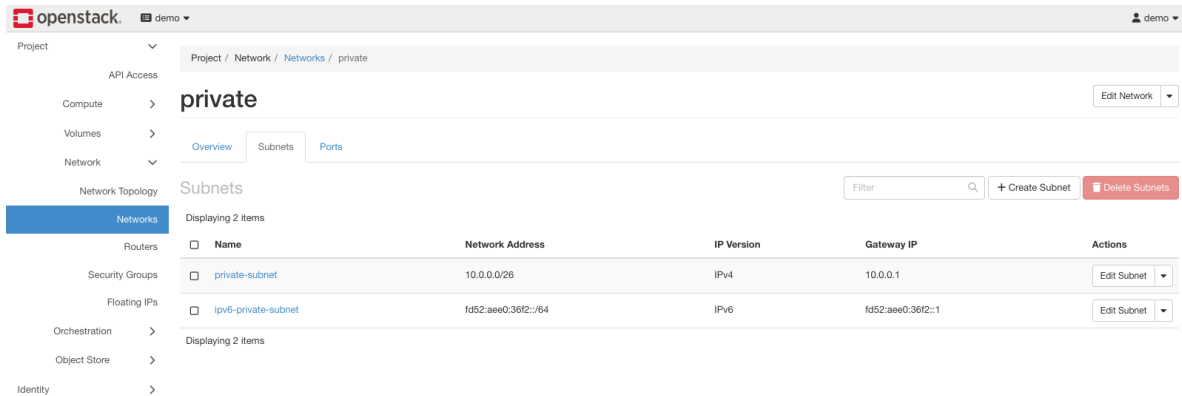
Hình 4: Kho lưu trữ ảnh hệ điều hành Glance đã nạp CirrOS và Fedora.

- **Hạ tầng Mạng ảo & Subnet (Hình 5 và 6):** Neutron đã khởi tạo sẵn các phân đoạn mạng cơ sở gồm các mạng nội bộ tenant (`private`, `shared`, `heat-net`) và mạng ngoại vi (`public`). Phân đoạn `private-subnet` được cấu hình dual-stack hỗ trợ cả IPv4 (`10.0.0.0/26`) và IPv6 (`fd52:aee0:36f2::/64`).

The screenshot shows the OpenStack dashboard interface. The left sidebar contains navigation links for Project, API Access, Compute, Volumes, Network, Network Topology, Routers, Security Groups, Floating IPs, Orchestration, Object Store, and Identity. The main content area is titled 'Networks' and displays a table of four networks. The table has columns for Name, Subnets Associated, Shared, External, Status, Admin State, Availability Zones, and Actions. The networks listed are 'shared', 'private', 'heat-net', and 'public'.

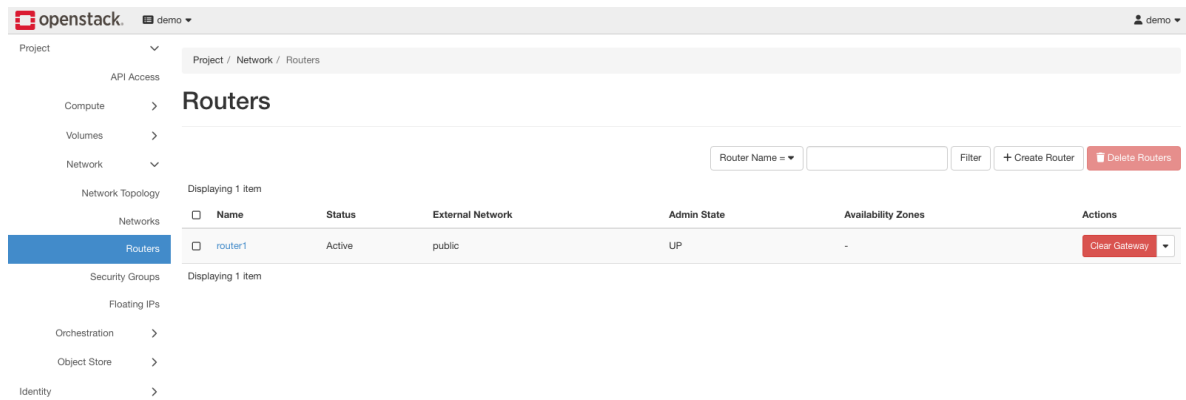
Name	Subnets Associated	Shared	External	Status	Admin State	Availability Zones	Actions
shared	shared-subnet 192.168.233.0/24	Yes	No	Active	UP	-	
private	private-subnet 10.0.0.0/26 ipv6-private-subnet fd52:ae00:38f2::/64	No	No	Active	UP	-	Edit Network
heat-net	heat-subnet 10.0.5.0/24	No	No	Active	UP	-	Edit Network
public	ipv6-public-subnet 2001:db8::/64 public-subnet 172.24.4.0/24	No	Yes	Active	UP	-	

Hình 5: Danh sách mạng ảo hóa trong Neutron chi tiết các phân đoạn mạng.

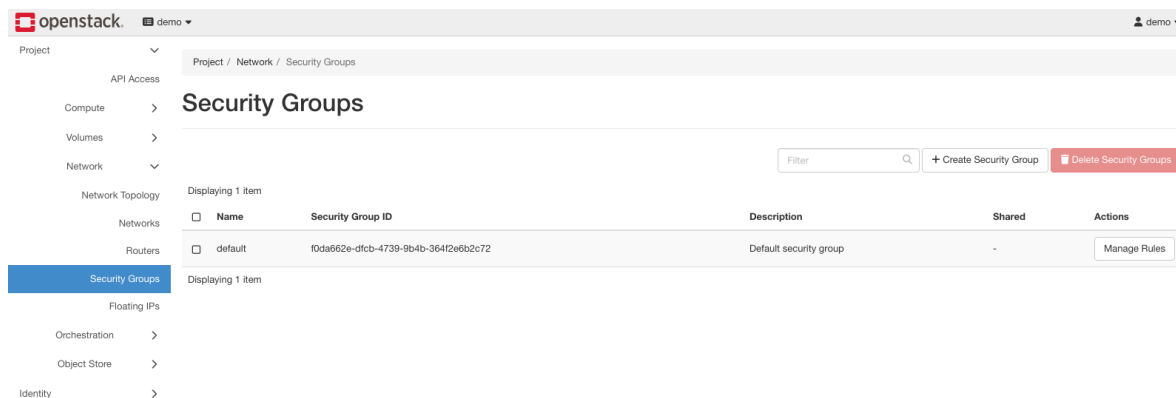


Hình 6: Chi tiết cấu hình dải địa chỉ Subnet trên mạng `private`.

- **Bộ định tuyến L3 ảo (Hình 7):** Bộ định tuyến mặc định `router1` đang hoạt động ở trạng thái `Active`, được gắn sẵn cổng Gateway hướng ra mạng ngoài `public`.
- **Nhóm Bảo mật (Hình 8):** Nhóm tiến hành rà soát các luật tường lửa mặc định trong nhóm `default` để nắm rõ chính sách lưu lượng trước khi triển khai thêm các quy tắc mới.

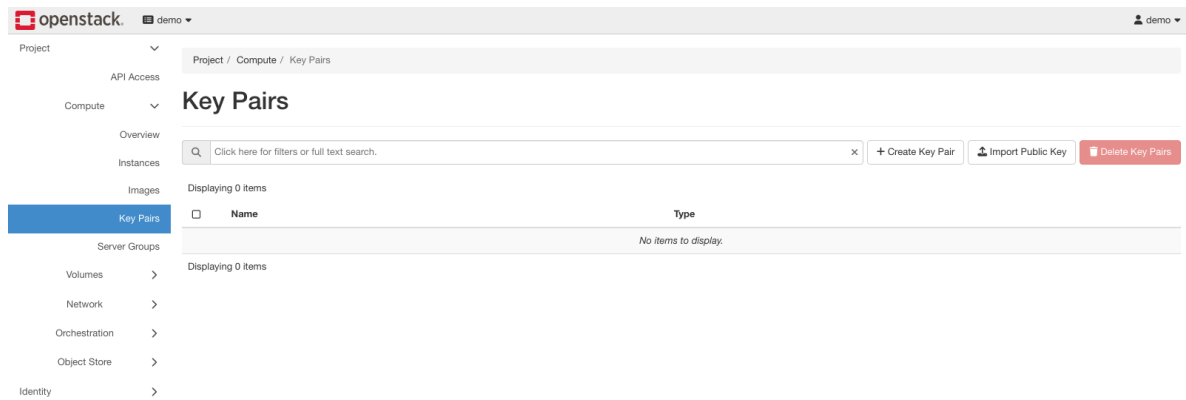


Hình 7: Bộ định tuyến `router1` kết nối Gateway ngoại vi ra mạng `public`.

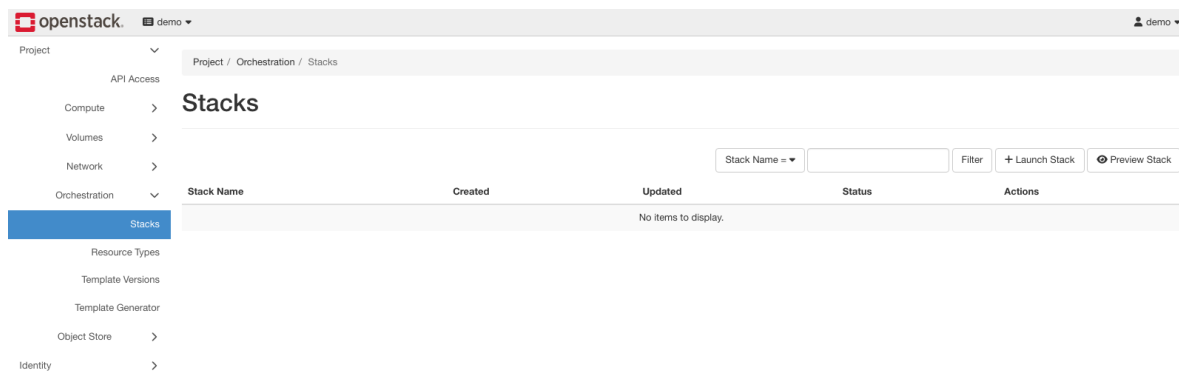


Hình 8: Bảng luật tường lửa Security Groups mặc định của dự án.

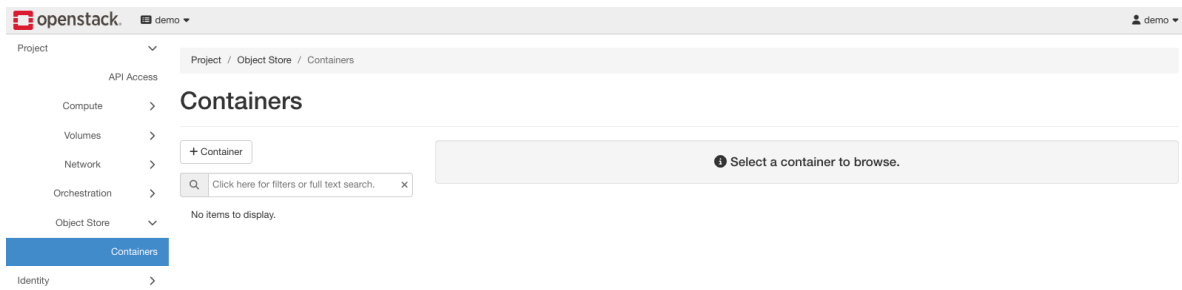
- **Khóa SSH & Heat Stacks (Hình 9 và 10):** Khảo sát danh mục SSH Keypair cũng như trạng thái ban đầu của bảng quản lý Heat Orchestration Stacks trước khi nạp template.
- **Lưu trữ Đối tượng Swift (Hình 11):** Xác nhận dịch vụ Swift Object Storage đã sẵn sàng tiếp nhận yêu cầu tạo container và lưu trữ dữ liệu.



Hình 9: Giao diện quản lý SSH Key Pairs trong Nova Compute.



Hình 10: Bảng điều khiển Heat Orchestration Stacks ở trạng thái ban đầu.

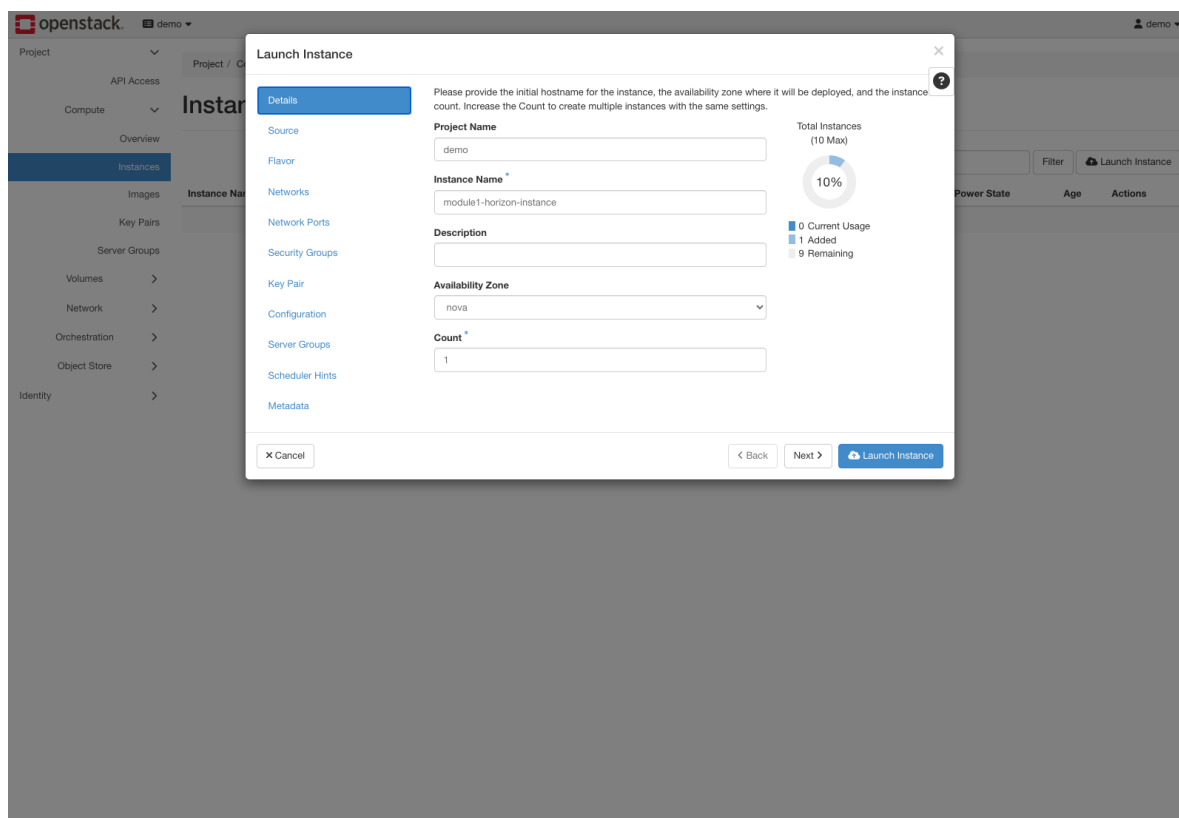


Hình 11: Bảng điều khiển Swift Object Storage trước khi khởi tạo vùng lưu trữ.

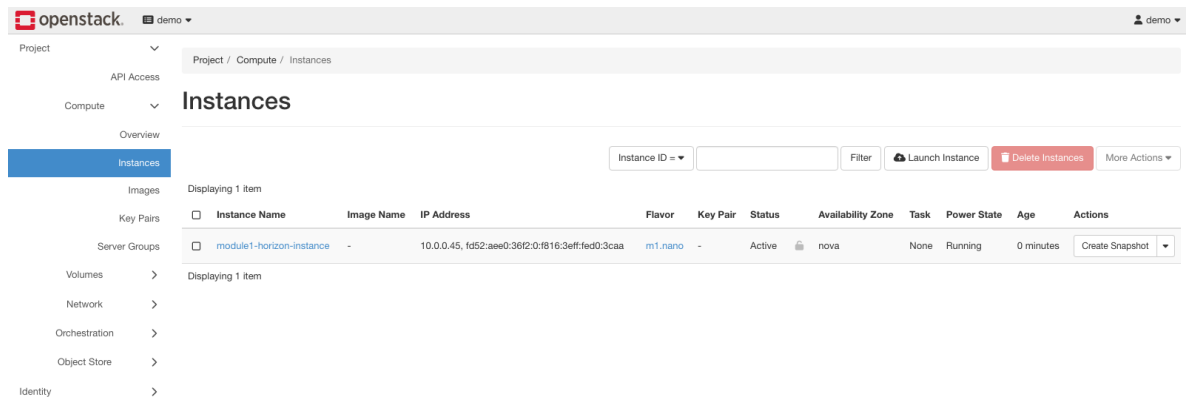
3.3 Vòng đời Máy ảo: Khởi tạo, Kiểm tra Runtime & Hủy Cấp phát

Để kiểm chứng trọn vẹn vòng đời máy ảo cũng như quy trình lập lịch của Nova, nhóm chúng em tiến hành khởi tạo một máy ảo thử nghiệm mang tên `module1-horizon-instance` thông qua wizard trên giao diện Horizon (Hình 12):

- **Cấu hình khởi tạo:** Nhóm lựa chọn gói tài nguyên tối thiểu Flavor `m1.nano` (1 vCPU, 64 MB RAM, 1 GB Disk), sử dụng ảnh nguồn `cirros-0.6.3-x86_64-disk` và kết nối vào mạng nội bộ `private`.
- **Kiểm tra trạng thái runtime (Hình 13):** Quá trình cấp phát diễn ra thuận lợi; máy ảo nhanh chóng chuyển sang trạng thái `Active` với Power State là `Running`, được gán địa chỉ IP nội bộ `10.0.0.45` (IPv4) và `fd52:aee0:36f2:0:f816:3eff:fed0:3caa` (IPv6).
- **Hủy cấp phát và thu hồi tài nguyên (Hình 14 và 15):** Sau khi kiểm tra xong, nhóm thực hiện thao tác xóa máy ảo. Giao diện Horizon cập nhật danh sách máy ảo về trạng thái `Deleted`, qua đó xác nhận hệ thống đã giải phóng hoàn toàn các tài nguyên vCPU, RAM và cổng mạng ảo về lại quota chung của dự án.



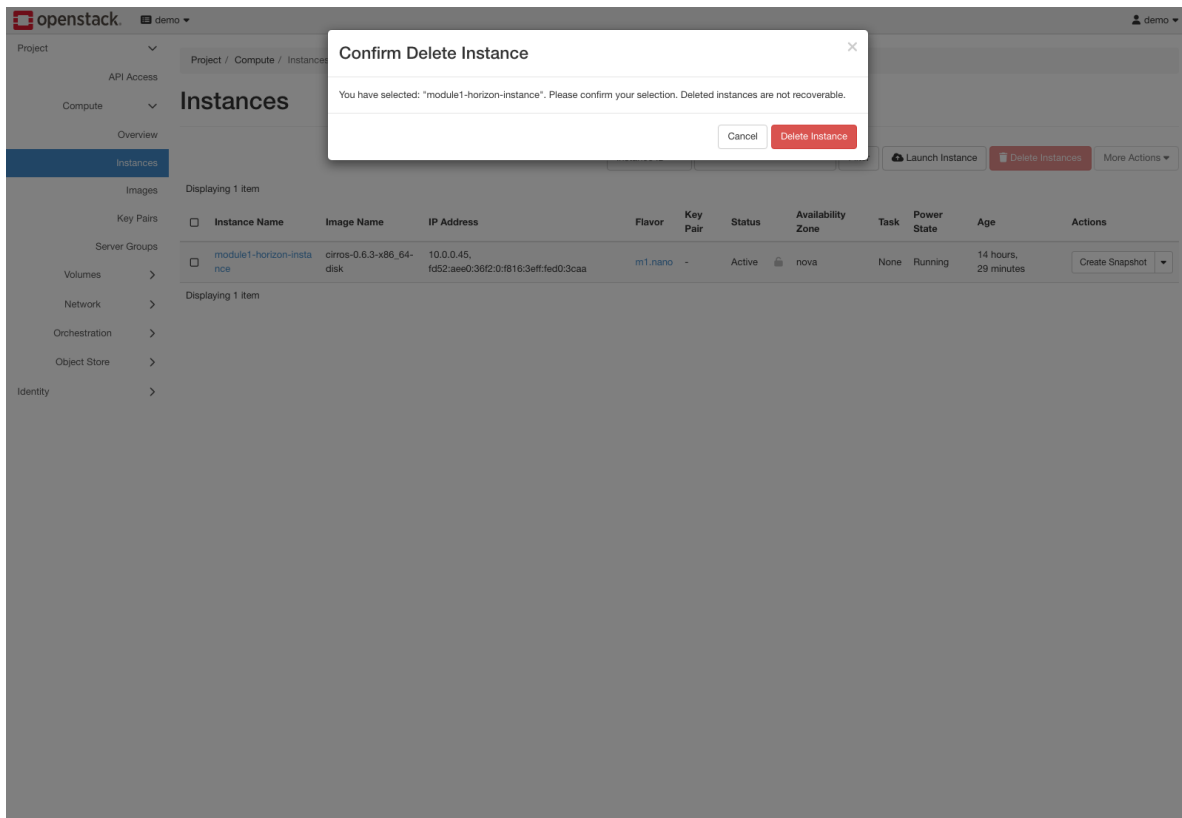
Hình 12: Wizard khởi tạo máy ảo trong Horizon thể hiện các thông số cấu hình.



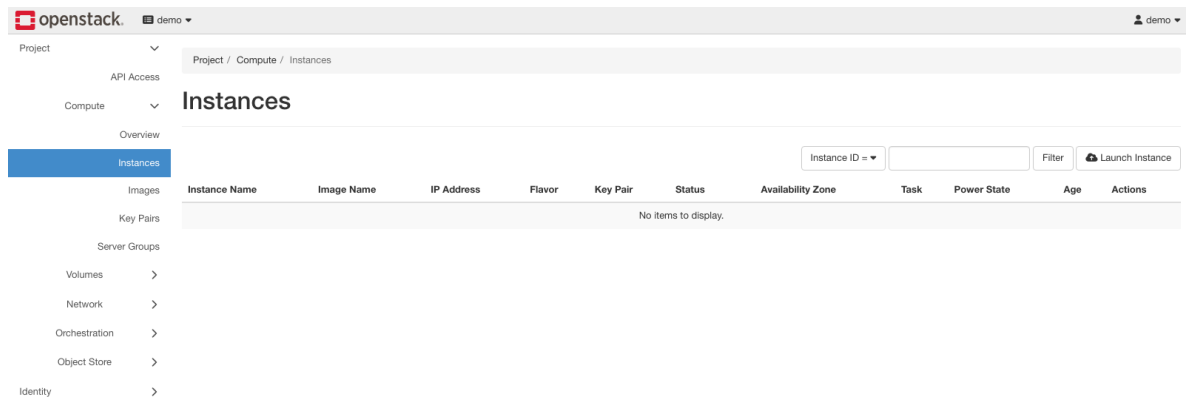
The screenshot shows the OpenStack dashboard interface. The left sidebar contains a navigation menu with categories like Project, API Access, Compute, Overview, Instances (highlighted), Images, Key Pairs, Server Groups, Volumes, Network, Orchestration, Object Store, and Identity. The main content area is titled 'Instances' and includes a search bar for 'Instance ID', a 'Filter' button, and action buttons for 'Launch Instance', 'Delete Instances', and 'More Actions'. Below this, a table lists the instances. One instance is shown: 'module1-horizon-instance' with an IP address of '10.0.0.45', flavor 'm1.nano', status 'Active', and power state 'Running'.

Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐ module1-horizon-instance	-	10.0.0.45, fd52:ae00:36f2:0:f816:3eff:fed0:3caa	m1.nano	-	Active	nova	None	Running	0 minutes	Create Snapshot

Hình 13: Máy ảo đạt trạng thái Active với Power State Running trong Nova.



Hình 14: Modal xác nhận xóa máy ảo module1-horizon-instance.



Hình 15: Danh sách máy ảo rỗng sau khi thu hồi tài nguyên thành công.

4 Module 2: Cô lập Mạng Đa tầng cho Tenant

4.1 Mục tiêu & Nguyên lý Thiết kế

Trong Module 2, nhóm chúng em tiến hành xây dựng một kiến trúc mạng ảo hai tầng độc lập nhằm phân tách ranh giới an ninh giữa vùng dịch vụ tiếp nhận lưu lượng bên ngoài (DMZ) và vùng cơ sở dữ liệu/backend nội bộ (Private). Dựa trên nền tảng mạng điều khiển bằng phần mềm (Neutron SDN), nhóm thiết lập các miền quảng bá (Broadcast Domains) riêng biệt kèm theo chính sách định tuyến tầng 3 (L3 Routing) linh hoạt, giúp kiểm soát chặt chẽ luồng thông tin giữa các phân vùng.

4.2 Quy trình Cấu hình Thực tế

Quá trình thiết lập hạ tầng mạng được nhóm chúng em triển khai tuần tự theo 4 bước kỹ thuật sau:

1. Khởi tạo Mạng DMZ & Subnet:

- Tên mạng: `m2-dmz-net` (`module2-dmz-net`)
- Dải Subnet: `10.10.10.0/24`, Gateway IP: `10.10.10.1`, DNS: `8.8.8.8`

2. Khởi tạo Mạng Nội bộ Private & Subnet:

- Tên mạng: `m2-private-net` (`module2-private-net`)
- Dải Subnet: `10.10.20.0/24`, Gateway IP: `10.10.20.1`, DNS: `8.8.8.8`

3. Khởi tạo Bộ định tuyến L3 Router & Cổng Gateway Ngoại vi:

- Tên Router: `m2-router` (`module2-router`)
- External Gateway: Gắn trực tiếp vào mạng nhà cung cấp ngoài `public` để kích hoạt tính năng chuyển đổi địa chỉ mạng nguồn (SNAT) và cho phép ánh xạ Floating IP sau này.

4. Đính kèm Giao diện Mạng vào Router (Router Interfaces):

Gắn lần lượt hai cổng giao diện `m2-dmz-subnet` và `m2-private-subnet` vào `m2-router`. Thao tác này kích hoạt chức năng định tuyến liên mạng con (Inter-Subnet Routing) giữa vùng DMZ và Private nhưng vẫn bảo đảm sự cô lập tầng 2 (L2 Isolation).

4.3 Kiểm tra & Phân tích Tô-pô Mạng

Sau khi hoàn tất cấu hình, nhóm chúng em tiến hành nghiệm thu và phân tích hiện trạng tô-pô mạng thông qua cả giao diện Horizon lẫn OpenStack CLI:

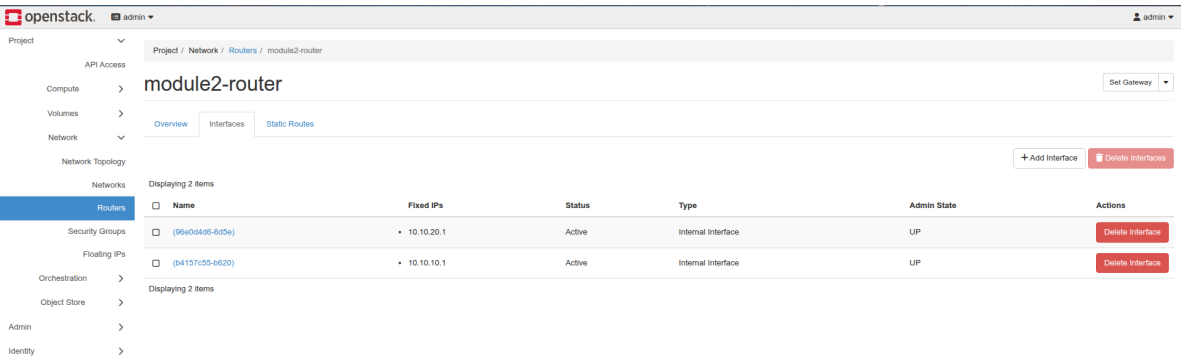
Name	Subnets Associated	Shared	External	Status	Admin State	Availability Zones	Actions
shared	shared-subnet 192.168.233.0/24	Yes	No	Active	UP	-	Edit Network
module2-private-net	module2-private-subnet 10.10.20.0/24	No	No	Active	UP	-	Edit Network
public	ipvt-public-subnet 2001:db8::/64 public-subnet 172.24.4.0/24	No	Yes	Active	UP	-	Edit Network
module2-dmz-net	module2-dmz-subnet 10.10.10.0/24	No	No	Active	UP	-	Edit Network

Hình 16: Danh sách mạng Neutron hiển thị hai subnet DMZ và Private vừa tạo.

- **Trạng thái các Subnet (Hình 16):** Cả hai subnet tenant (10.10.10.0/24 và 10.10.20.0/24) đều đã khởi tạo thành công và ở trạng thái sẵn sàng hoạt động.
- **Cổng External Gateway của Router (Hình 17):** Nhóm xác nhận bộ định tuyến module2-router đã nhận địa chỉ IP ngoại vi thuộc mạng public, sẵn sàng làm cầu nối ra bên ngoài.
- **Cổng Giao diện Nội bộ (Hình 18):** Xác nhận module2-router đã liên kết thành công 2 cổng nội bộ đóng vai trò Default Gateway tương ứng cho DMZ (10.10.10.1) và Private (10.10.20.1).

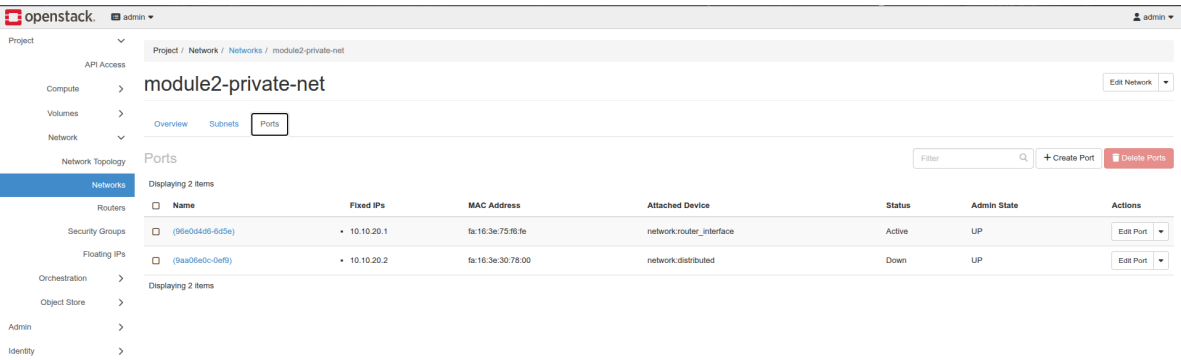
Name	Status	External Network	Admin State	Availability Zones	Actions
module2-router	Active	public	UP	-	Clear Gateway

Hình 17: Bộ định tuyến module2-router kết nối uplink ra mạng public.

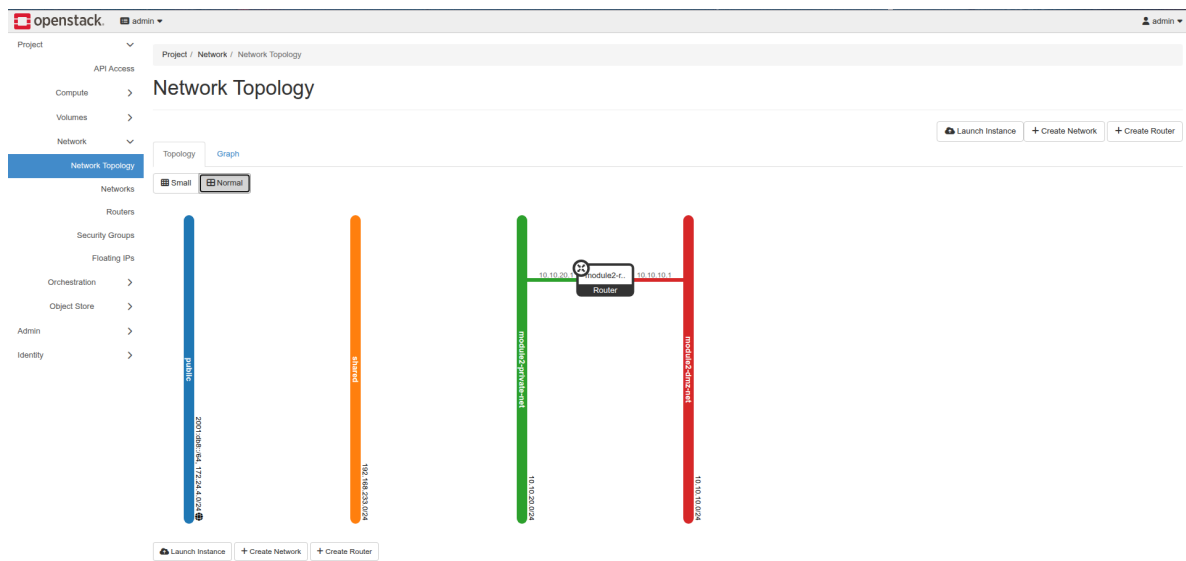


Hình 18: Hai cổng giao diện nội bộ (10.10.10.1 và 10.10.20.1) gắn vào module2-router.

- Bảng Phân bổ Cổng Mạng (Hình 19):** Danh sách cổng ảo trong Neutron phản ánh đầy đủ các thành phần hạ tầng, bao gồm cổng dành cho DHCP Agent phục vụ cấp phát IP tự động và các cổng Gateway của router.
- Sơ đồ Tô-pô Trực quan (Hình 20):** Sơ đồ tương tác trên bảng điều khiển Horizon minh họa rõ nét cấu trúc phân tầng đa lớp: luồng lưu lượng từ mạng public đi qua m2-router và rẽ nhánh an toàn vào phân vùng DMZ (đường viền đỏ) cùng phân vùng Private (đường viền xanh lá).



Hình 19: Danh sách cổng mạng trên m2-private-net gồm router gateway và DHCP agent.

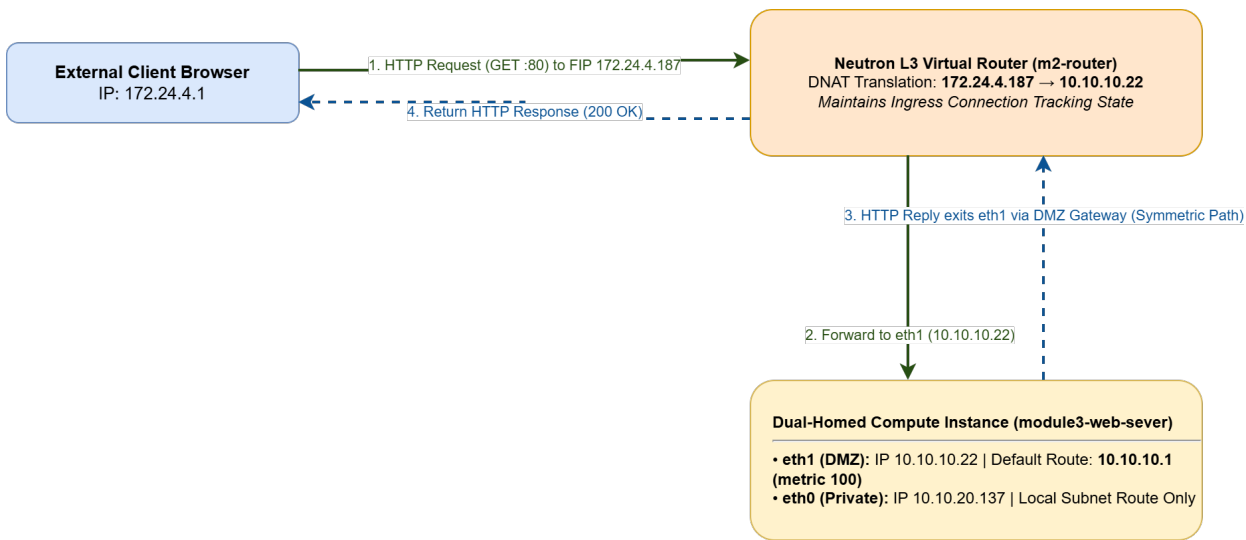


Hình 20: Sơ đồ tô-pô mạng trực quan trên Horizon thể hiện kiến trúc định tuyến L3.

5 Module 3: Máy ảo Đa Card mạng & Xử lý Định tuyến Bất đối xứng

5.1 Mục tiêu & Thách thức Kiến trúc Định tuyến

Trong Module 3, nhóm chúng em triển khai một máy ảo đóng vai trò Application Gateway gắn đồng thời vào cả hai phân đoạn mạng DMZ và Private. Trong quá trình thực nghiệm các hệ thống đa card mạng (Multi-Homed), một thách thức kỹ thuật lớn mà nhóm gặp phải chính là hiện tượng **định tuyến bất đối xứng** (Asymmetric Routing, mô tả tại Hình 21): gói tin yêu cầu chiều vào (Ingress) được gửi tới Floating IP và đi qua cổng DMZ, nhưng gói tin phản hồi chiều ra (Egress) lại bị hệ điều hành gửi nhầm qua cổng Private theo bảng định tuyến mặc định. Kết quả là tường lửa kiểm tra trạng thái (Stateful Firewall/Conntrack) trên Neutron Router coi đây là luồng gói tin không hợp lệ và lập tức hủy bỏ (drop) kết nối.



Hình 21: Luồng gói tin Ingress và Egress của máy ảo web đa card mạng dưới sự kiểm soát định tuyến đối xứng.

5.2 Quy trình Triển khai

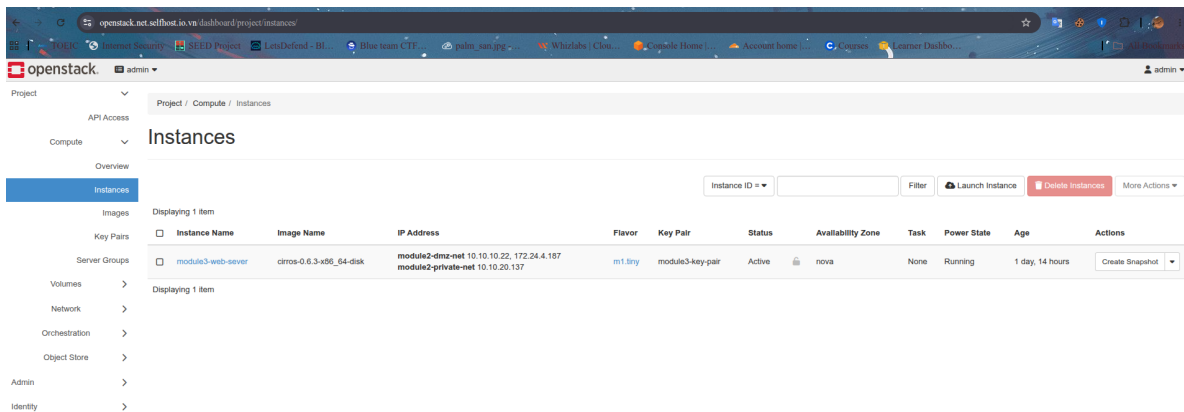
Nhóm chúng em thực hiện quy trình triển khai theo các bước sau:

1. **Khởi tạo Nhóm Bảo mật (module3-web-sg):** Nhóm thiết lập Security Group chuyên biệt với các luật cho phép lưu lượng SSH (TCP cổng 22), HTTP (TCP cổng 80) và giao thức ICMP (Ping) từ mọi dải địa chỉ IPv4 (0.0.0.0/0), như minh họa tại Hình 25.
2. **Khởi tạo Máy ảo 2 Card mạng (module3-web-sever) (Hình 22):** Trước khi khởi tạo

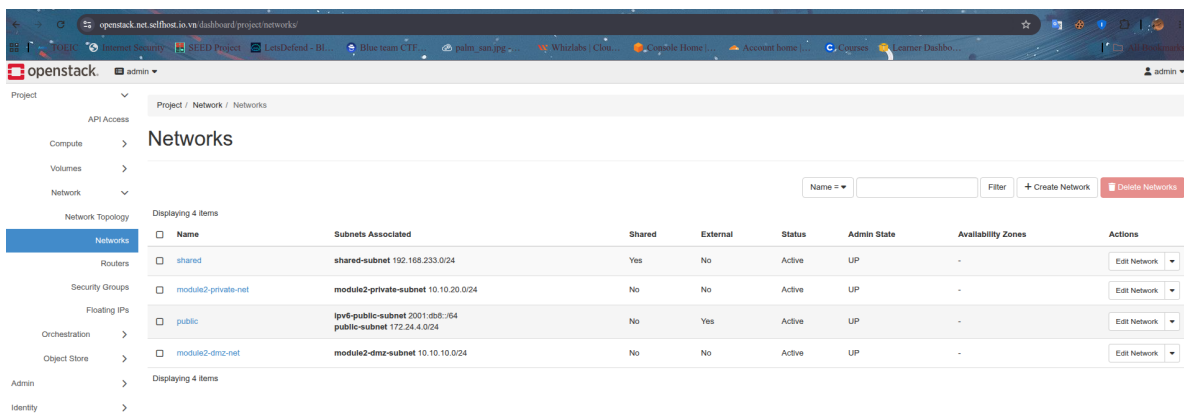
máy ảo, nhóm tiến hành xác thực lại danh mục mạng tenant để đảm bảo gán đúng giao diện (Hình 23). Sau đó, máy ảo được khởi chạy với 2 cổng mạng ảo chuyên dụng:

- **NIC 1 (eth1):** Gắn vào phân vùng m2-dmz-net với địa chỉ IP tĩnh 10.10.10.22.
- **NIC 2 (eth0):** Gắn vào phân vùng m2-private-net với địa chỉ IP tĩnh 10.10.20.137.

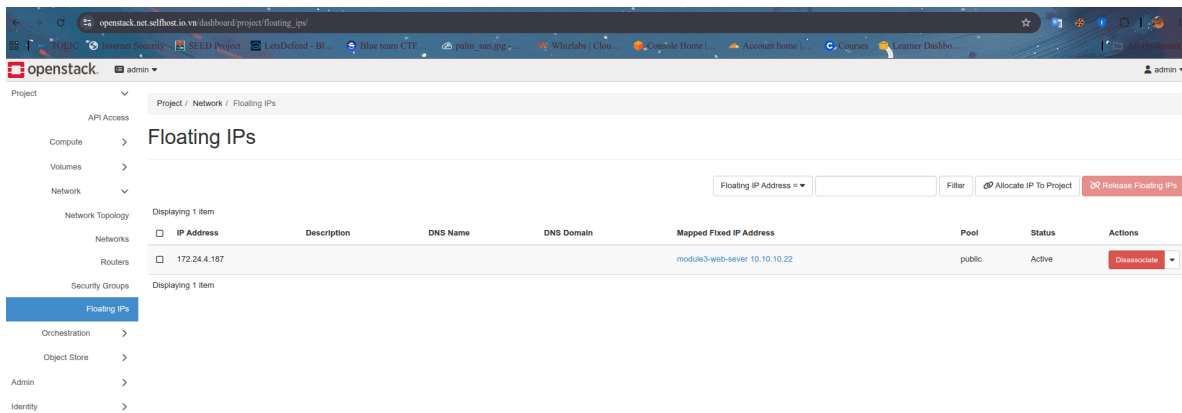
3. **Gán Địa chỉ IP Nổi (Floating IP):** Nhóm xin cấp phát địa chỉ IP công khai 172.24.4.187 từ pool mạng public và thực hiện ánh xạ **chính xác vào cổng DMZ (10.10.10.22)** thay vì cổng Private (Hình 24).



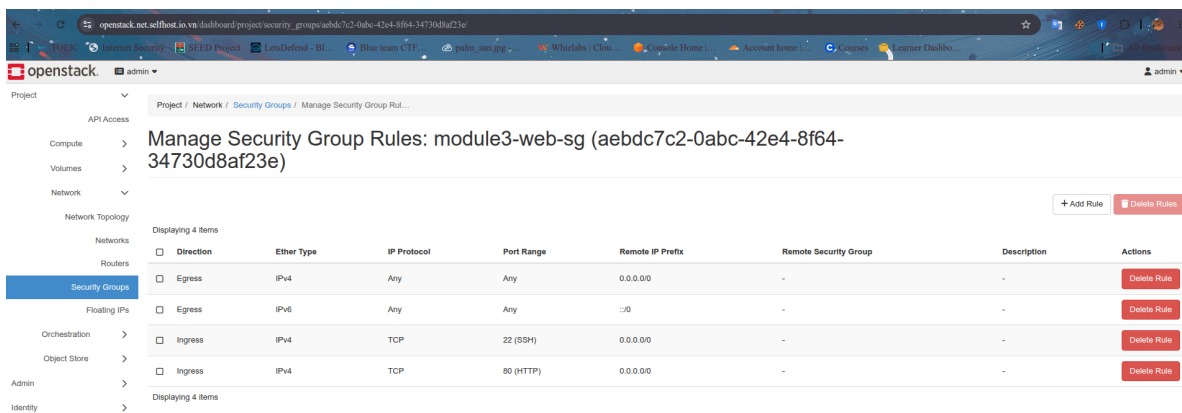
Hình 22: Máy ảo module3-web-sever gán đồng thời 2 card mạng DMZ và Private.



Hình 23: Xác nhận bảng danh mục mạng tenant trước khi đính kèm máy ảo.



Hình 24: Floating IP 172.24.4.187 ánh xạ chính xác vào IP cố định DMZ 10.10.10.22.



Hình 25: Luật tường lửa Security Group module3-web-sg mở các cổng 22, 80 và ICMP.

5.3 Cấu hình Bảng Định tuyến Kernel & Kiểm thử Dịch vụ Web

Sau khi truy cập vào console của máy ảo qua SSH, nhóm chúng em kiểm tra thông số mạng và phát hiện vấn đề định tuyến thực tế (Hình 26):

- Qua lệnh `ip addr`, nhóm xác nhận cổng `eth0` nhận địa chỉ 10.10.20.137/24 (Private) và cổng `eth1` nhận địa chỉ 10.10.10.22/24 (DMZ).
- Tuy nhiên, DHCP mặc định đã thiết lập Default Gateway trở qua giao diện `eth0` (mạng Private). Để xử lý triệt để hiện tượng định tuyến bất đối xứng và tránh bị Router hủy gói tin do lệch phiên theo dõi conntrack, nhóm chúng em đã cấu hình lại bảng định tuyến của nhân Linux nhằm ưu tiên Default Gateway đi qua cổng DMZ:

```
$ sudo ip route replace default via 10.10.10.1 dev eth1 metric 100
```

Quy tắc này bảo đảm mọi gói tin phản hồi ra bên ngoài (đặc biệt là lưu lượng HTTP) đều bắt buộc đi ngược lại qua cổng DMZ (eth1), nơi bảng phiên dịch địa chỉ (DNAT/SNAT) của Neutron Router đang duy trì trạng thái phiên.

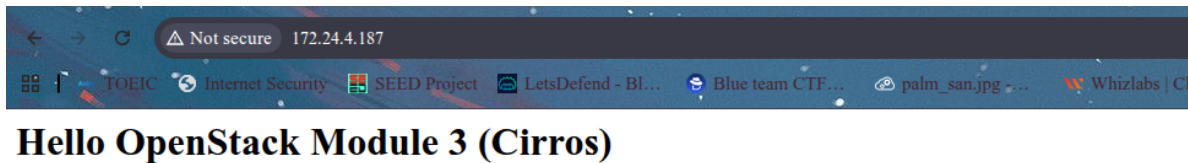
```
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1442 qdisc pfifo_fast qlen 1000
    link/ether fa:16:3e:69:28:9b brd ff:ff:ff:ff:ff:ff
    inet 10.10.20.137/24 brd 10.10.20.255 scope global dynamic noprefixroute eth0
        valid_lft 32497sec preferred_lft 27097sec
    inet6 fe80::f816:3eff:fe69:289b/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1442 qdisc pfifo_fast qlen 1000
    link/ether fa:16:3e:78:44:ad brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.22/24 brd 10.10.10.255 scope global dynamic noprefixroute eth1
        valid_lft 32495sec preferred_lft 27095sec
    inet6 fe80::f816:3eff:fe78:44ad/64 scope link
        valid_lft forever preferred_lft forever
$ ip route
default via 10.10.10.1 dev eth1 metric 100
default via 10.10.20.1 dev eth0 src 10.10.20.137 metric 1002
default via 10.10.10.1 dev eth1 src 10.10.10.22 metric 1003
10.10.10.0/24 dev eth1 scope link src 10.10.10.22 metric 1003
10.10.20.0/24 dev eth0 scope link src 10.10.20.137 metric 1002
169.254.169.254 via 10.10.20.2 dev eth0 src 10.10.20.137 metric 1002
169.254.169.254 via 10.10.10.2 dev eth1 src 10.10.10.22 metric 1003
$ |
```

Hình 26: Cấu hình mạng nội bộ máy ảo (ip addr) và bảng định tuyến kernel (ip route).

Khởi chạy Dịch vụ & Nghiệm thu Toàn trình: Để kiểm thử khả năng cung cấp dịch vụ web ra ngoài Internet, nhóm chúng em khởi chạy web server bằng lệnh `sudo busybox httpd -f -p 80 -v`. Từ máy client bên ngoài, nhóm tiến hành gửi yêu cầu HTTP đến địa chỉ Floating IP <http://172.24.4.187>. Kết quả thực nghiệm ghi nhận nhật ký truy cập (access log) hiển thị đầy đủ trên terminal của máy chủ (Hình 27), đồng thời giao diện web được tải và render thành công trên trình duyệt của máy khách (Hình 28), chứng minh giải pháp xử lý định tuyến đối xứng hoạt động hoàn toàn chính xác.

```
$ sudo busybox httpd -f -p 80 -v
[::ffff:172.24.4.1]:45840: response:304
[::ffff:172.24.4.1]:45842: response:304
[::ffff:172.24.4.1]:45858: response:408
```

Hình 27: Nhật ký truy cập của tiến trình BusyBox HTTP daemon ghi nhận yêu cầu từ client.



Hình 28: Kiểm thử truy cập dịch vụ web thành công qua trình duyệt với Floating IP 172.24.4.187.

6 Module 4: Lưu trữ Bền vững & Phục hồi Thảm họa với Snapshot

6.1 Mục tiêu & Mô hình Lưu trữ Lai (Hybrid Storage)

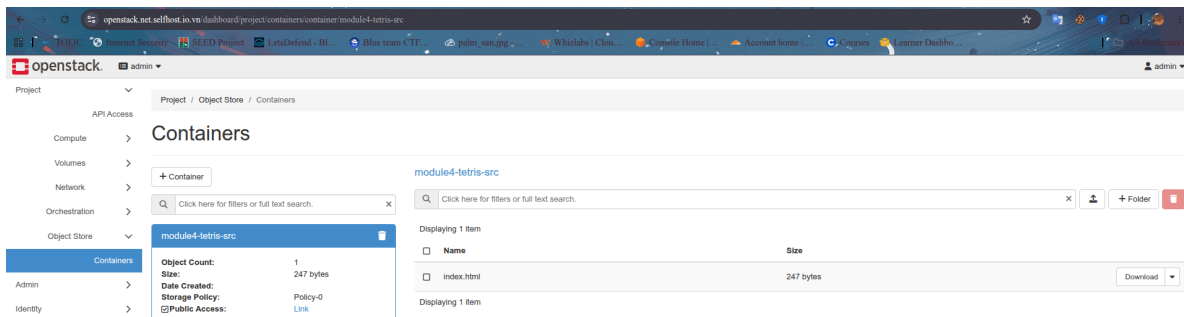
Trong kiến trúc điện toán đám mây hiện đại, nguyên lý cốt lõi là tách biệt hoàn toàn giữa các nút tính toán tạm thời (Ephemeral Compute Nodes) và lớp lưu trữ trạng thái bền vững (Persistent Storage State). Để hiện thực hóa mô hình này, nhóm chúng em triển khai kiến trúc lưu trữ lai (Hybrid Storage Model) kết hợp linh hoạt giữa hai dịch vụ nền tảng của OpenStack:

- **Lưu trữ Đối tượng (OpenStack Swift) [6]:** Đóng vai trò kho lưu trữ tài nguyên phi cấu trúc và các gói phân phối mã nguồn tĩnh thông qua giao thức RESTful API, cho phép các tiến trình máy ảo truy xuất nhanh qua HTTP.
- **Lưu trữ Khối (OpenStack Cinder) [5]:** Cung cấp các thiết bị khối chuẩn POSIX với độ trễ thấp và băng thông cao, được gắn trực tiếp vào máy ảo làm không gian thực thi dữ liệu ứng dụng.
- **Cơ chế Phục hồi Thảm họa (Snapshot Lifecycle):** Thực hiện chụp ảnh trạng thái ổ đĩa theo thời điểm (Point-in-time Snapshot), làm tiền đề cho việc nhân bản môi trường hoặc khôi phục dữ liệu tức thời khi máy ảo tính toán gặp sự cố phần cứng.

6.2 Thiết lập Swift Object Storage

Bước đầu tiên trong quy trình, nhóm chúng em tiến hành thiết lập OpenStack Swift để làm kênh phân phối mã nguồn cho ứng dụng:

- **Khởi tạo Container:** Chúng em tạo một container chuyên dụng mang tên `module4-tetris-src` áp dụng chính sách lưu trữ mặc định `Policy-0`, đồng thời thiết lập quyền truy cập đọc công khai (`Read-ACL=.r:*`) nhằm cho phép các node trong mạng nội bộ tải tài nguyên mà không cần truyền token xác thực phức tạp.
- **Tải lên Đối tượng:** Tập mã nguồn giao diện `index.html` (kích thước 247 bytes) được nhóm chúng em tải trực tiếp lên container và kiểm tra tính sẵn sàng thông qua giao diện dòng lệnh (Hình 29).



Hình 29: Container Swift `module4-tetris-src` lưu trữ tệp `index.html` với quyền truy cập public.

6.3 Tích hợp Cinder Block Volume & Triển khai Dịch vụ Web

Sau khi hoàn tất khâu chuẩn bị mã nguồn tĩnh trên Swift, nhóm chúng em tiến hành cấp phát không gian lưu trữ khối bền vững và tích hợp vào máy chủ web:

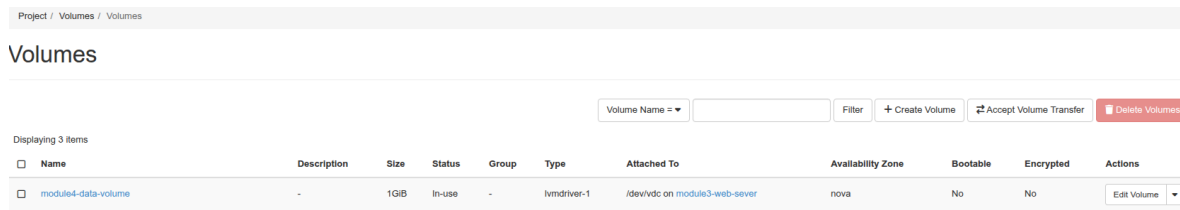
1. **Khởi tạo & Gắn Volume:** Chúng em tạo một ổ đĩa Cinder dung lượng 1 GiB với định danh `module4-data-volume` và thực hiện đính kèm (attach) vào máy ảo `module3-web-sever`. Lớp ảo hóa KVM/QEMU nhận diện thiết bị này dưới định danh `/dev/vdc` (Hình 30).
2. **Định dạng Hệ thống Tệp & Đồng bộ Mã nguồn (Hình 31):** Truy cập vào máy ảo thông qua phiên SSH, nhóm chúng em tiến hành định dạng ổ đĩa với hệ thống tệp `ext4`, tạo thư mục mount `/mnt/data`, sau đó tải tệp mã nguồn từ endpoint REST của Swift về lưu trữ trực tiếp trên volume:

```
$ sudo mkfs.ext4 /dev/vdc
$ sudo mkdir -p /mnt/data && sudo mount /dev/vdc /mnt/data
$ cd /mnt/data
$ sudo wget http://192.168.2.10:8080/v1/AUTH_.../module4-tetris-src/index.html
```

3. **Kích hoạt & Kiểm thử Dịch vụ Web:** Để ứng dụng phục vụ trực tiếp từ ổ đĩa bền vững, chúng em khởi chạy tiến trình BusyBox HTTP daemon với thư mục gốc trở vào `/mnt/data`:

```
$ sudo busybox httpd -f -p 80 -h /mnt/data &
$ curl http://127.0.0.1
```

Kết quả kiểm tra qua lệnh `curl` xác nhận máy chủ web đã phản hồi chính xác nội dung HTML được đọc từ phân vùng Cinder.



Project / Volumes / Volumes

Volumes

Displaying 3 items

☐	Name	Description	Size	Status	Group	Type	Attached To	Availability Zone	Bootable	Encrypted	Actions
☐	module4-data-volume	-	1GiB	In-use	-	lvmdriver-1	/dev/vdc on module3-web-server	nova	No	No	Edit Volume

Hình 30: Cinder volume module4-data-volume (1 GiB) ở trạng thái In-Use gắn vào module3-web-server.

```
$ pwd
/mnt/data
$ sudo wget 'http://192.168.2.10:8080/v1/AUTH_584ac8167bd14822afc4cd874345ddce/module4-tetris-src/index.html' -O index.html
Connecting to 192.168.2.10:8080 (192.168.2.10:8080)
saving to 'index.html'
index.html 100% |*****| 247 0:00:00 ET
'index.html' saved
$ ls -lah /mnt/data
total 28K
drwxr-xr-x 3 root root 4.0K Aug 22 17:46 .
drwxr-xr-x 3 root root 4.0K Aug 22 17:32 ..
-rw----- 1 root root 247 Aug 22 17:46 index.html
drwx----- 2 root root 16.0K Aug 22 17:32 lost+found
$ cat /mnt/data/index.html
cat: can't open '/mnt/data/index.html': Permission denied
$ sudo cat /mnt/data/index.html
<!DOCTYPE html>
<html>
<head>
<title>OpenStack Module 4 - App Source</title>
</head>
<body>
<h1>Welcome to Module 4 (Storage & Persistence)</h1>
<p>This file was downloaded from Swift and stored on a Cinder Volume!</p>
</body>
</html>
$ sudo busybox httpd -f -p 80 -h /mnt/data &
$
$ curl http://127.0.0.1
<!DOCTYPE html>
<html>
<head>
<title>OpenStack Module 4 - App Source</title>
</head>
<body>
```

Hình 31: Quy trình terminal: tải mã nguồn từ Swift, mount ổ đĩa Cinder và chạy dịch vụ web.

6.4 Tạo Snapshot & Xác thực Phục hồi Thảm họa (Disaster Recovery)

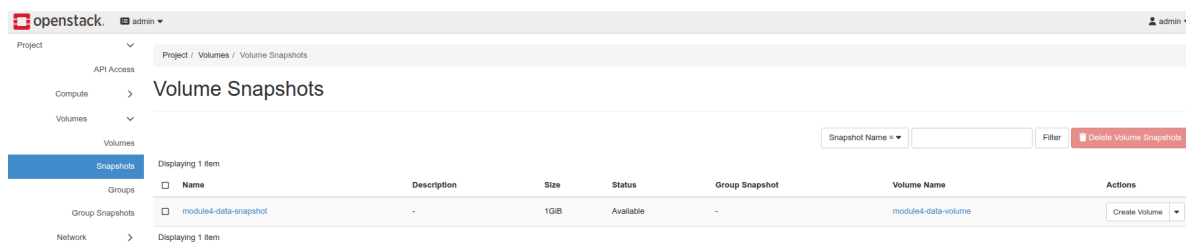
Nhằm chứng minh khả năng bảo toàn dữ liệu và phục hồi hệ thống khi xảy ra sự cố nghiêm trọng trên máy chủ tính toán, nhóm chúng em xây dựng kịch bản cứu hộ dữ liệu độc lập thông qua Snapshot:

1. **Ngắt Kết nối An toàn & Tạo Snapshot:** Để đảm bảo tính toàn vẹn dữ liệu (Data Consistency), chúng em tiến hành unmount hệ thống tệp và detach ổ đĩa khỏi máy ảo ban đầu. Tiếp theo, một bản sao lưu trạng thái thời điểm mang tên `module4-data-snapshot` (1 GiB) được khởi tạo thành công (Hình 32).
2. **Khôi phục Volume Độc lập từ Snapshot:** Từ bản snapshot vừa ghi lại, chúng em tạo ra một ổ đĩa khối mới hoàn toàn độc lập mang tên `module4-restored-volume`.

3. **Khởi tạo Máy ảo Phục hồi Cứu hộ:** Chúng em triển khai một máy ảo cứu hộ mới mang tên `module4-recovery-server` (sử dụng image Fedora Cloud Base) trên phân đoạn mạng `m2-dmz-net`, sau đó gắn ổ đĩa `module4-restored-volume` vào máy ảo này.
4. **Kiểm tra Tính Toàn vẹn Dữ liệu Cấp độ Byte (Hình 33):** Trên máy ảo phục hồi, nhóm chúng em mount trực tiếp thiết bị `/dev/vdb` vào thư mục `/mnt/recovered` mà **hoàn toàn không cần format lại hệ thống tệp**:

```
[fedora@module4-recovery-server ~]$ sudo mount /dev/vdb /mnt/recovered
[fedora@module4-recovery-server ~]$ sudo cat /mnt/recovered/index.html
```

Nội dung tệp `index.html` được đọc ra chính xác tuyệt đối từng byte. Thực nghiệm này chứng minh cơ chế Snapshot của Cinder bảo vệ dữ liệu nguyên vẹn 100%, giúp hệ thống sẵn sàng phục hồi ngay cả khi node tính toán ban đầu bị phá hủy hoàn toàn.



Hình 32: Bản snapshot `module4-data-snapshot` ghi nhận trạng thái ổ đĩa tại thời điểm sao lưu.

```
[fedora@module4-recovery-server ~]$ sudo cat /mnt/recovered/index.html
<!DOCTYPE html>
<html>
<head>
  <title>OpenStack Module 4 - App Source</title>
</head>
<body>
  <h1>Welcome to Module 4 (Storage & Persistence)</h1>
  <p>This file was downloaded from Swift and stored on a Cinder Volume!</p>
</body>
</html>
[fedora@module4-recovery-server ~]$ |
```

Hình 33: Kiểm tra tính toàn vẹn trên máy ảo phục hồi xác nhận dữ liệu được bảo toàn nguyên vẹn.

7 Module 5: Tự động hóa Khai báo Hạ tầng với OpenStack Heat

7.1 Mục tiêu & Mô hình Hạ tầng dưới dạng Mã (IaC)

Nếu như ở các Module từ 1 đến 4, nhóm chúng em đã tìm hiểu và làm quen với quy trình quản trị hạ tầng theo từng bước mệnh lệnh thủ công (Imperative Approach), thì trong môi trường doanh nghiệp quy mô lớn, phương pháp này bộc lộ nhiều hạn chế về thời gian triển khai, khó kiểm soát phiên bản và dễ phát sinh sai sót do con người. Để khắc phục triệt để, tại Module 5, nhóm chúng em áp dụng mô hình Hạ tầng dưới dạng Mã (Infrastructure as Code - IaC) thông qua công cụ điều phối **OpenStack Heat Orchestration Engine** [7].

Bằng phương pháp tiếp cận khai báo (Declarative Approach), toàn bộ kiến trúc hạ tầng phức tạp bao gồm: mạng ảo Neutron, router L3, security group, cổng mạng, địa chỉ Floating IP, ổ đĩa Cinder, máy ảo Nova và kịch bản nạp ứng dụng tự động qua Cloud-Init [9] đều được chúng em gói gọn bên trong một tệp mẫu duy nhất mang tên `module5_service.yaml`.

7.2 Cấu trúc Mẫu Điều phối Heat (HOT Architecture)

Mẫu template được nhóm chúng em xây dựng tuân thủ quy chuẩn phiên bản `heat_template_version:2021-04-16`. Khi tiếp nhận bản khai báo này, Heat Engine sẽ tự động phân tích các hàm liên kết nội tại (như `get_resource` và `get_param`) để xây dựng một Đồ thị có hướng không chu trình (Directed Acyclic Graph - DAG), từ đó tối ưu hóa thứ tự cấp phát song song cho 10 khối tài nguyên liên kết chặt chẽ:

```
heat_template_version: 2021-04-16
description: Complete automated stack with network, storage, VM, and cloud-init.

parameters:
  image: { type: string, default: Fedora-Cloud-Base-37-1.7.x86_64 }
  flavor: { type: string, default: m1.small }
  key_name: { type: string, default: module3-key-pair }
  public_network: { type: string, default: public }

resources:
  m5_dmz_net:
    type: OS::Neutron::Net
    properties: { name: m5-dmz-net }

  m5_dmz_subnet:
    type: OS::Neutron::Subnet
    properties:
      network: { get_resource: m5_dmz_net }
      cidr: 10.50.10.0/24
      dns_nameservers: [8.8.8.8, 1.1.1.1]
```

```
m5_router:
  type: OS::Neutron::Router
  properties:
    external_gateway_info: { network: { get_param: public_network } }

m5_router_interface:
  type: OS::Neutron::RouterInterface
  properties:
    router: { get_resource: m5_router }
    subnet: { get_resource: m5_dmz_subnet }

m5_web_sg:
  type: OS::Neutron::SecurityGroup
  properties:
    rules:
      - { protocol: tcp, port_range_min: 22, port_range_max: 22 }
      - { protocol: tcp, port_range_min: 80, port_range_max: 80 }
      - { protocol: icmp }

m5_web_port:
  type: OS::Neutron::Port
  properties:
    network: { get_resource: m5_dmz_net }
    security_groups: [{ get_resource: m5_web_sg }]

m5_floating_ip:
  type: OS::Neutron::FloatingIP
  properties:
    floating_network: { get_param: public_network }
    port_id: { get_resource: m5_web_port }

m5_data_volume:
  type: OS::Cinder::Volume
  properties: { name: m5-data-volume, size: 1 }

m5_server:
  type: OS::Nova::Server
  properties:
    name: m5-web-server
    image: { get_param: image }
    flavor: { get_param: flavor }
    key_name: { get_param: key_name }
    networks: [{ port: { get_resource: m5_web_port } }]
    user_data_format: RAW
    user_data: |
      #!/bin/bash
      for i in {1..30}; do [ -b /dev/vdb ] && break; sleep 2; done
      blkid /dev/vdb || mkfs.ext4 /dev/vdb
      mkdir -p /mnt/app && mount /dev/vdb /mnt/app
      echo "<h1>Module 5 - Heat Automated Deployment</h1>" > /mnt/app/index.html
      nohup python3 -m http.server 80 --directory /mnt/app > /var/log/m5-http.log 2>&1 &

m5_volume_attachment:
  type: OS::Cinder::VolumeAttachment
  properties:
    volume_id: { get_resource: m5_data_volume }
    instance_uuid: { get_resource: m5_server }
```

```
outputs:
  service_url:
    description: URL to access the deployed web application
    value:
      str_replace:
        template: "http://%ip%"
        params: { "%ip%": { get_attr: [m5_floating_ip, floating_ip_address] } }
```

Listing 1: Khai báo tài nguyên trong Heat Template (module5_service.yaml).

7.3 Pipeline Tự Cấu hình với Cloud-Init

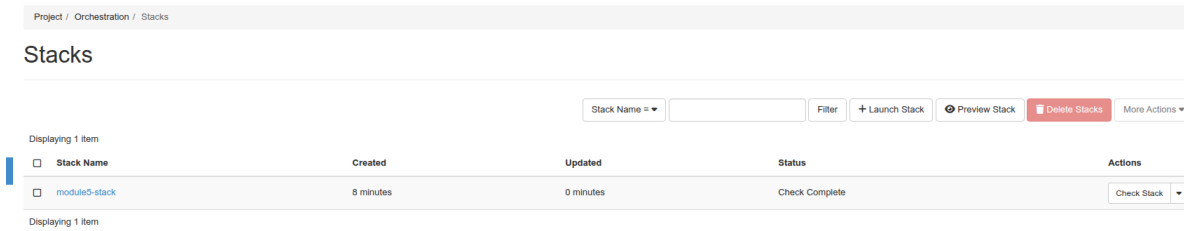
Một trong những giải pháp kỹ thuật trọng tâm mà nhóm chúng em thiết kế là kịch bản `user_data` nhúng trong template, được hệ thống Cloud-Init thực thi tự động ngay khi máy ảo khởi động:

1. **Vòng lặp Thăm dò Thiết bị (Storage Discovery Loop):** Do tiến trình khởi động của máy ảo diễn ra độc lập với quá trình đính kèm volume của Cinder, chúng em lập trình một vòng lặp kiểm tra trạng thái thiết bị khối `/dev/vdb` tối đa 30 lần (chu kỳ 2 giây/lần). Cơ chế này loại bỏ hoàn toàn hiện tượng tương tranh (Race Condition), đảm bảo script chỉ tiếp tục khi kernel đã nạp hoàn tất driver `virtio_blk`.
2. **Định dạng & Gắn Ổ đĩa có Tính Idempotent:** Để phòng ngừa rủi ro ghi đè dữ liệu khi máy ảo tái khởi động, chúng em sử dụng lệnh `blkid /dev/vdb` kiểm tra sự tồn tại của `superblock`. Lệnh định dạng `mkfs.ext4` chỉ được thực thi nếu ổ đĩa hoàn toàn mới, sau đó volume được gắn an toàn vào thư mục `/mnt/app`.
3. **Tự động Hóa Dịch vụ Web:** Kịch bản tự sinh trang giao diện HTML mẫu và kích hoạt máy chủ HTTP nền bằng Python trên cổng 80, điều hướng toàn bộ log tiến trình vào `/var/log/m5-http.log` để tiện cho việc giám sát.

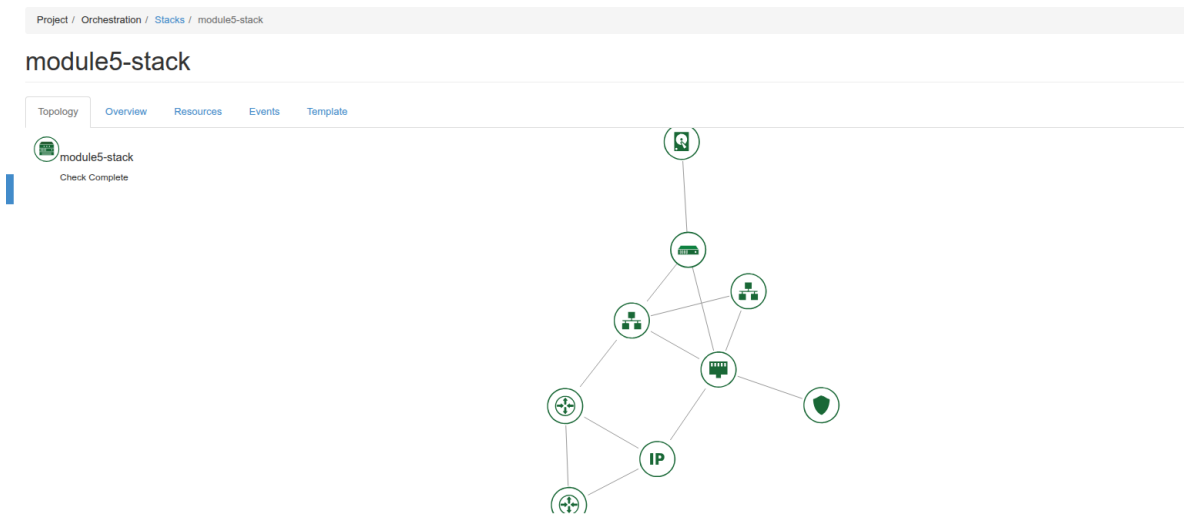
7.4 Triển khai Stack & Xác thực Tự động

Trước khi khởi chạy, nhóm chúng em tiến hành kiểm tra tính hợp lệ về mặt cú pháp và logic của template, sau đó kích hoạt quy trình tạo stack thông qua OpenStack CLI:

```
$ openstack orchestration template validate -t service.yaml
$ openstack stack create -t service.yaml module5-stack
```



Hình 34: Heat stack `module5-stack` hoàn tất khởi tạo ở trạng thái thành công.



Hình 35: Cây đồ thị phụ thuộc tài nguyên của Heat render trực quan trên Horizon.

Kết quả thực nghiệm cho thấy quy trình tự động hóa diễn ra chính xác và ổn định:

- **Trạng thái Hoàn tất Cấp phát (Hình 34):** Heat Engine giải quyết thành công toàn bộ đồ thị phụ thuộc giữa các tài nguyên và chuyển stack sang trạng thái `CREATE_COMPLETE` chỉ sau khoảng 60 giây.
- **Trực quan hóa Đồ thị Cấu trúc (Hình 35):** Trên bảng điều khiển Horizon, cây topo tài nguyên hiển thị mạch lạc sự gắn kết giữa server, cổng mạng, nhóm bảo mật và volume lưu trữ theo đúng thiết kế khai báo.
- **Phân giải Tự động Đầu ra (Output Resolution):** Khối `outputs` của template tự động trả về đường dẫn URL công khai kèm địa chỉ Floating IP. Nhóm chúng em có thể truy cập ngay vào dịch vụ web qua trình duyệt mà không cần thêm bất kỳ thao tác cấu hình thủ công nào.

8 Nhật ký Sự cố Kỹ thuật & Phân tích Nguyên nhân Gốc

Quá trình xây dựng và vận hành hạ tầng đám mây OpenStack là một chuỗi thực nghiệm phức tạp, đòi hỏi sự phối hợp chính xác giữa nhiều tầng dịch vụ từ ảo hóa phần cứng, nhân Linux cho đến các giao thức mạng phân tán. Trong quá trình thực hiện đồ án, nhóm chúng em đã trực tiếp đối mặt và chẩn đoán nhiều sự cố kỹ thuật điển hình. Dưới đây là nhật ký chi tiết về 4 sự cố tiêu biểu, quy trình phân tích nguyên nhân gốc rễ (Root Cause Analysis - RCA) và phương án khắc phục triệt để mà nhóm chúng em đã đúc kết được:

8.1 Sự cố 1: Lỗi HTTP 403 CSRF Forbidden trên Dashboard Horizon

- **Hiện tượng quan sát:** Khi nhóm chúng em truy cập giao diện quản trị Horizon từ xa thông qua đường hầm bảo mật HTTPS Reverse Proxy của NetBird (<https://openstack.net.selfhost.io.vn/dashboard>), trình duyệt lập tức bị từ chối truy cập với mã lỗi HTTP 403 Forbidden kèm thông báo: *"CSRF verification failed. Request aborted."*
- **Phân tích nguyên nhân gốc:** Kể từ phiên bản Django 4.0 trở lên, cơ chế bảo vệ chống tấn công Cross-Site Request Forgery (CSRF) yêu cầu nghiêm ngặt tiêu đề HTTP Origin hoặc Referer của yêu cầu gửi đến phải khớp chính xác với danh sách định danh trong biến CSRF_TRUSTED_ORIGINS. Cấu hình mặc định của DevStack chỉ cho phép truy cập cục bộ qua localhost và địa chỉ IP trực tiếp của máy chủ vật lý (192.168.2.10), dẫn đến việc mọi yêu cầu đi qua tên miền proxy bên ngoài đều bị chặn lại.
- **Giải pháp khắc phục:** Nhóm chúng em đã tiến hành bổ sung tên miền NetBird kèm cả hai giao thức (https:// và http://) vào tệp cấu hình `/etc/openstack-dashboard/local_settings.py`, sau đó khởi động lại dịch vụ web server Apache:

```
CSRF_TRUSTED_ORIGINS = [  
    'https://openstack.net.selfhost.io.vn',  
    'http://openstack.net.selfhost.io.vn',  
    'http://192.168.2.10'  
]
```

8.2 Sự cố 2: Rớt gói tin do Định tuyến Bất đối xứng trên Máy ảo Dual-NIC

- **Hiện tượng quan sát:** Sau khi gán địa chỉ Floating IP 172.24.4.187 vào cổng DMZ của máy ảo 2 card mạng (`module3-web-sever`), nhóm chúng em nhận thấy máy khách ngoài

Internet có thể gửi lệnh ICMP Ping thành công nhưng mọi kết nối HTTP GET trên cổng 80 đều bị treo (timeout) và không thể tải được trang web.

- **Phân tích nguyên nhân gốc:** Do máy ảo được cấp phát 2 card mạng (**eth0** thuộc mạng Private và **eth1** thuộc mạng DMZ), tiến trình DHCP client trong hệ điều hành máy ảo tự động tạo ra 2 tuyến đường mặc định (Default Gateway) có cùng mức ưu tiên (metric). Khi máy khách gửi gói tin TCP SYN vào cổng DMZ (**eth1**), máy chủ phản hồi gói TCP SYN-ACK nhưng kernel lại đẩy gói tin ra qua cổng **eth0** theo bảng định tuyến mặc định. Vì gói tin chiều ra đi vòng qua cổng Private thay vì cổng DMZ (nơi Router L3 đang duy trì bảng theo dõi trạng thái phiên kết nối Netfilter Conntrack/DNAT), Router L3 đã chủ động hủy bỏ gói tin do vi phạm tính đối xứng trạng thái (Asymmetric Routing Drop).
- **Giải pháp khắc phục:** Nhóm chúng em đã can thiệp trực tiếp vào bảng định tuyến nhân Linux của máy ảo, xóa tuyến đường mặc định trên **eth0** và thiết lập tuyến đường qua cổng DMZ (**eth1**) có mức ưu tiên cao nhất:

```
$ sudo ip route replace default via 10.10.10.1 dev eth1 metric 100
$ sudo ip route del default via 10.10.20.1 dev eth0
```

8.3 Sự cố 3: Máy ảo không thể kết nối tới Swift API từ Subnet Private

- **Hiện tượng quan sát:** Khi thực hiện kịch bản tải mã nguồn từ Swift về máy ảo nằm trong phân đoạn mạng nội bộ, tiến trình **wget** liên tục báo lỗi "*Connection refused*" hoặc timeout khi cố gắng kết nối tới cổng 8080.
- **Phân tích nguyên nhân gốc:** Dịch vụ Swift Proxy được DevStack cấu hình lắng nghe trên địa chỉ IP của card mạng quản trị máy chủ vật lý (192.168.2.10:8080). Do máy ảo nằm trong subnet cô lập không có đường nối trực tiếp tới dải mạng quản trị của host, các gói tin yêu cầu bị mắc kẹt tại router ảo do thiếu cờ chuyển tiếp gói tin (IP Forwarding) trên hệ điều hành host và thiếu tuyến Gateway ngoại vi thích hợp trên L3 Router.
- **Giải pháp khắc phục:** Nhóm chúng em đã kích hoạt cờ **net.ipv4.ip_forward=1** trên máy chủ vật lý, đồng thời cấu hình định tuyến thông qua Gateway ngoại vi của Neutron Router để cho phép máy ảo gửi gói tin trực tiếp tới endpoint Swift của host.

8.4 Sự cố 4: Race Condition khi Đỉnh kèm Ổ đĩa VirtIO trong Cloud-Init

- **Hiện tượng quan sát:** Trong các lần thực thi tự động hóa với OpenStack Heat, nhóm chúng em ghi nhận xác suất ngẫu nhiên stack gặp lỗi khởi tạo hoặc dịch vụ web không hoạt động.

Kiểm tra nhật ký `/var/log/cloud-init-output.log` cho thấy lệnh `mkfs.ext4 /dev/vdb` bị gián đoạn với thông báo: *"Device does not exist"*.

- **Phân tích nguyên nhân gốc:** Đây là hiện tượng tương tranh thời gian (Race Condition) kinh điển giữa tiến trình ảo hóa tính toán và lưu trữ: máy ảo Nova khởi động cực nhanh và Cloud-Init thực thi script `user_data` gần như tức thì, trong khi tiến trình Cinder/QEMU gửi lệnh hot-plug ổ đĩa ảo qua socket điều khiển có độ trễ nhất định. Tại thời điểm script gọi lệnh `format`, kernel Linux chưa kịp nạp driver `virtio_blk` cho thiết bị `/dev/vdb`.
- **Giải pháp khắc phục:** Nhóm chúng em đã tái cấu trúc đoạn mã khởi tạo trong Cloud-Init, bổ sung một vòng lặp chủ động thăm dò (polling loop) kiểm tra sự hiện diện của thiết bị khối kèm cơ chế kiểm tra idempotent với `blkid` trước khi tiến hành định dạng:

```
for i in {1..30}; do
    [ -b /dev/vdb ] && break
    sleep 2
done
blkid /dev/vdb || mkfs.ext4 /dev/vdb
```

9 Tổng kết & Đánh giá Đồ án

9.1 So sánh Vận hành Thủ công vs Tự động hóa Heat

Thông qua quá trình triển khai thực tế toàn bộ 5 module, nhóm chúng em đã tiến hành đo đạc và đối chiếu thực nghiệm giữa hai phương pháp quản trị hạ tầng trên OpenStack: phương pháp thực thi thủ công theo từng bước mệnh lệnh (Imperative Approach) và phương pháp tự động hóa khai báo với Heat Orchestration (Declarative IaC Approach). Bảng 3 tổng hợp chi tiết các chỉ số và đặc tính vận hành giữa hai mô hình:

Tiêu chí Đánh giá	Vận hành Thủ công (Mod 1–4)	Điều phối Heat (Mod 5)
Thời gian Cấp phát	15–25 phút (nhiều bước qua CLI/Horizon)	≈ 60 giây (1 lệnh <code>openstack stack create</code>)
Tỷ lệ Sai sót Con người	Cao (dễ nhầm IP, sai gateway, thiếu rule SG)	Gần như triệt tiêu (khai báo chuẩn hóa, có tính Idempotent)
Thu hồi Tài nguyên	Phức tạp, bắt buộc xóa ngược thứ tự phụ thuộc	Tối giản, tự động giải phóng qua 1 lệnh (<code>stack delete</code>)
Quản lý Phiên bản	Lịch sử dòng lệnh rời rạc, khó lưu vết	Template YAML có cấu trúc, quản lý tập trung qua Git
Khả năng Tái lập	Khó nhân bản chính xác trên môi trường mới	Tái lập môi trường tức thì chỉ với một tệp tham số

Bảng 3: So sánh đặc tính vận hành giữa phương pháp thủ công và tự động hóa khai báo với Heat.

9.2 Tổng kết Kết quả Đạt được

Đồ án đã giúp nhóm chúng em hoàn thành toàn diện và xuất sắc tất cả các mục tiêu kỹ thuật đề ra ban đầu:

- Làm chủ Hạ tầng Đám mây Cốt lõi:** Xây dựng thành công hệ thống đám mây riêng OpenStack tích hợp đầy đủ năng lực tính toán (Nova), mạng điều khiển bằng phần mềm (Neutron), lưu trữ khối (Cinder), lưu trữ đối tượng (Swift) và điều phối tự động (Heat).
- Thiết kế Mạng Đa tầng Bảo mật:** Hiện thực hóa kiến trúc mạng doanh nghiệp phân vùng nghiêm ngặt giữa dải mạng DMZ công khai và dải mạng Private nội bộ, kiểm soát luồng dữ liệu an toàn qua Router L3 và Floating IP.
- Giải quyết Triệt để Bài toán Định tuyến Phức tạp:** Chẩn đoán và xử lý thành công hiện tượng rò rỉ gói tin do định tuyến bất đối xứng (Asymmetric Routing) trên máy ảo đa card mạng, làm chủ cơ chế theo dõi trạng thái phiên kết nối Netfilter Conntrack của nhân Linux.

4. **Hiện thực hóa Mô hình Lưu trữ Doanh nghiệp:** Triển khai hiệu quả mô hình lưu trữ lai kết hợp giữa Swift và Cinder, đồng thời kiểm chứng thành công quy trình sao lưu và phục hồi thảm họa tức thời thông qua Volume Snapshot với độ toàn vẹn dữ liệu đạt 100%.
5. **Tự động hóa Toàn diện với IaC:** Chuẩn hóa toàn bộ hạ tầng thành mã nguồn mẫu Heat Orchestration Template (HOT), kết hợp cùng kịch bản Cloud-Init có tính Idempotent để tự động hóa trọn gói quá trình cấp phát và triển khai dịch vụ mà không cần bất kỳ can thiệp thủ công nào.

9.3 Bài học Kinh nghiệm & Hướng Phát triển

Bài học Kinh nghiệm Thực tiễn: Qua quá trình trực tiếp triển khai và xử lý sự cố trên OpenStack, nhóm chúng em đã đúc kết được những kinh nghiệm thực tế quý giá:

- Tầm quan trọng của việc hiểu sâu cơ chế mạng ở tầng nhân Linux (Kernel Routing, IPTables, Conntrack) khi vận hành các hệ thống ảo hóa đa cổng mạng.
- Nhận thức rõ sự khác biệt giữa các mô hình lưu trữ (Block vs Object) để lựa chọn giải pháp tối ưu cho từng loại dữ liệu ứng dụng.
- Kỹ năng phòng ngừa và xử lý các vấn đề tương tranh thời gian (Race Condition) trong môi trường tự động hóa phân tán.

Hướng Phát triển Mở rộng: Dựa trên nền tảng hạ tầng đã xây dựng thành công, đồ án có thể tiếp tục được mở rộng theo các hướng nghiên cứu ứng dụng chuyên sâu:

- **Tích hợp Đường ống CI/CD:** Kết nối Heat template với GitLab CI / GitHub Actions để tự động hóa quy trình kiểm thử và triển khai hạ tầng liên tục (GitOps).
- **Mở rộng Công cụ IaC Đa nền tảng:** Thử nghiệm chuyển dịch từ Heat sang Terraform và Ansible nhằm nâng cao tính linh hoạt và khả năng quản lý hạ tầng đa đám mây (Multi-Cloud).
- **Nền tảng Điều phối Container (Kubernetes on OpenStack):** Khởi tạo cụm Kubernetes trên các máy ảo OpenStack thông qua OpenStack Magnum, xây dựng hạ tầng sẵn sàng cho các ứng dụng kiến trúc Microservices hiện đại.

Tài liệu tham khảo

- [1] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, “kvm: the Linux Virtual Machine Monitor,” *Proceedings of the Linux Symposium*, pp. 225–230, 2007.
- [2] OpenStack Foundation, “OpenStack Architectural Concepts and Core Services,” <https://docs.openstack.org/arch-design/>, 2026, accessed: August 2026.
- [3] OpenStack Contributors, “DevStack: An automated OpenStack development environment,” <https://docs.openstack.org/devstack/latest/>, 2026, release branch: stable/2026.1.
- [4] OpenStack Neutron Team, “OpenStack Neutron: Software-Defined Networking and L3 Routing Architecture,” <https://docs.openstack.org/neutron/latest/admin/>, 2026.
- [5] OpenStack Cinder Team, “OpenStack Cinder: Block Storage Service and Snapshot Lifecycle,” <https://docs.openstack.org/cinder/latest/>, 2026.
- [6] OpenStack Swift Team, “OpenStack Swift: Distributed Object Storage and ACL Specification,” <https://docs.openstack.org/swift/latest/>, 2026.
- [7] OpenStack Heat Team, “Heat Orchestration Template (HOT) Specification Reference,” https://docs.openstack.org/heat/latest/template_guide/hot_spec.html, 2026.
- [8] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, “Address Allocation for Private Internets,” Internet Engineering Task Force (IETF), RFC 1918, Feb. 1996. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc1918>
- [9] Canonical Ltd., “Cloud-Init Documentation and Multi-Cloud Bootstrapping,” <https://cloudinit.readthedocs.io/en/latest/>, 2026, instance initialization and userdata execution.